

ATLAS

Contrato integral de implementação

Da Alpha 2 ao funcionário digital generalista

Preparado para **Vinícius Milanez**
Versão 2.0 · 24 de setembro de 2026
Repositório: ViniciusMilanez82/atlas
Baseline: v0.1.0-alpha.2 · d81cece

31 capítulos + 3 anexos obrigatórios

32 ordens de correção · 24 pacotes de produto
40 cenários de aceite · 56 itens de backlog

Instalar. Criar o funcionário. Conversar. Delegar. Receber.
Com identidade, memória, contas, arquivos e ambiente próprios.

Este documento prescreve o que construir, como integrar e como comprovar. Não declara o Atlas pronto, não autoriza gastos e não substitui testes em Mac real.

Sumário

Os anexos fazem parte do contrato. O Markdown do pacote é a fonte textual prioritária para a IA programadora.

0. Como usar este contrato	4
1. Visão exata do produto e limites	4
2. Definição de produto concluído	6
3. Arquitetura de referência e fronteiras reais	6
4. Contratos, esquema de dados e migrações	7
5. Recebimento durável, idempotência e eventos	8
6. Controle imediato, tarefas e recuperação	9
7. Conversa natural, continuidade e identidade	11
8. Memória duradoura, temporal e aprendizado	12
9. Privacidade, classificação e saída de dados	13
10. Documentos: compreender, não apenas receber arquivos	14
11. Modelos de IA e configuração de inteligência	15
12. Orçamento, credenciais e autorizações	16
13. Planejamento, ferramentas e conclusão comprovada	17
14. Arquivos, artefatos e qualidade visual	18
15. O computador próprio do Atlas	19
16. Navegador, pesquisa e superação legítima de obstáculos	20
17. Instalação de ferramentas e aprendizado de skills	21
18. Contas, e-mail e identidade operacional	22
19. Canal remoto pelo celular	23
20. Voz: a mesma conversa, sem um segundo agente	23
21. Interface e instalação para pessoa comum	24
22. Ciclo de vida, disponibilidade e recuperação do Mac	25

23. Proteção de armazenamento, backup e atualização	26
24. Observabilidade, metas e critérios mensuráveis	26
25. Testes que realmente representam o produto	27
26. Plano de construção e gates de conclusão	28
27. Como a IA deve trabalhar e prestar contas	29
28. Evidência de aceite e entrega final	30
29. Configuração de referência e contratos operacionais	30
30. Referências e controle de fontes	32
ANEXO A — 32 ordens de correção obrigatórias	34
ANEXO B — 40 cenários de aceite do produto	51
ANEXO C — 24 pacotes de implementação até a V1	60
Encerramento do contrato	67

0. Como usar este contrato

0.1 Instrução para a IA programadora

Leia este arquivo integralmente, incluindo os anexos. Trabalhe no repositório existente. Sua função é implementar, integrar, testar e produzir uma entrega demonstrável; não devolver apenas outra arquitetura ou uma lista de promessas. Utilize `PROMPT_INICIAR_IMPLEMENTACAO.txt` como instrução inicial e o backlog fornecido como índice de trabalho.

A branch principal foi consultada nesta preparação e continuava no commit indicado. Antes de alterar qualquer arquivo, confira o HEAD e as mudanças locais: este documento não autoriza reverter correções posteriores. Compare a situação atual com a baseline e registre diferenças. Os caminhos citados são referências à organização existente ou destinos propostos, não uma ordem para duplicar módulos.

0.2 Precedência e mudança de escopo

Este contrato detalha a visão original, incorpora os 32 achados da auditoria da Alpha 2 e refina decisões que ficaram incompletas. Preserve `docs/MASTER_SPEC.md` e os documentos históricos. Adicione esta versão em `docs/spec/`, registre uma ADR de adoção e aponte o contrato vigente no índice. Não altere o histórico para aparentar que um requisito sempre foi outro.

Em conflitos: respeite primeiro segurança e autorização; depois a visão e os requisitos deste contrato; depois contratos legados compatíveis; por último sugestões de bibliotecas e layouts. A auditoria descreve a baseline, não prova que uma falha continua existindo depois de novos commits. Contestar um achado exige reprodução ou inspeção rastreável, não opinião.

DEVE é obrigatório. **NÃO DEVE** é proibido. **DEVERIA** admite alternativa com ADR e evidência. **PODE** é opcional. Números de capacidade, latência e avaliação aqui definidos são metas de projeto a medir, não resultados já obtidos.

Alterar tecnologia interna sem mudar a experiência, o risco ou o custo material pode ser uma decisão técnica registrada. Retirar voz, ambiente próprio, memória, entrega remota ou generalidade exige decisão explícita do proprietário. Não converter limitação temporária em redução permanente de escopo.

0.3 Separar duas inteligências

A **IA programadora** lê este contrato e modifica o repositório. O **Atlas** é o produto que executará tarefas do proprietário. O prompt de construção não é o prompt de sistema do Atlas. A assinatura do editor/assistente de programação não é automaticamente uma credencial de inferência do aplicativo. As permissões para desenvolver também não autorizam o Atlas a comprar, enviar mensagens a terceiros ou acessar contas pessoais.

1. Visão exata do produto e limites

1.1 O que o usuário deve perceber

Uma única identidade contínua, chamada Atlas por padrão, com nome configurável. O usuário conversa como com um profissional: explica um objetivo, envia materiais, responde perguntas, muda de ideia e recebe o trabalho. O sistema escolhe passos e ferramentas; não exige que o usuário monte agentes, fluxos, pipelines ou comandos.

Exemplos de calendário, família, viagens, propostas e empresa são cenários para comprovar generalidade. Nenhum deles deve virar o domínio exclusivo do produto. A habilidade comum é

receber contexto, recordar, pesquisar, raciocinar, executar, conferir, comunicar e aprender procedimentos reutilizáveis.

O termo funcionário digital representa a experiência, não consciência, personalidade jurídica independente nem uma identidade humana falsa. Contas ficam sob controle administrativo de uma pessoa ou organização responsável. Não inventar documentos, idade, telefone, CPF, poderes de representação ou acesso a serviços.

1.2 O escritório pertence ao Atlas

O Atlas mantém seu próprio navegador, downloads, arquivos, agenda operacional, credenciais, histórico e memória. Não sincroniza automaticamente e-mail, fotos, contatos, histórico de navegação, pastas ou sessões do proprietário. Arquivos enviados entram como cópias explícitas. Dados de terceiros recebidos por meio dessas cópias continuam sujeitos a classificação e finalidade.

O computador pode ser dedicado ao Atlas, mas o aplicativo continua respeitando o sistema hospedeiro. Não bloquear a saída do proprietário, esconder serviços, desabilitar FileVault/Gatekeeper ou exigir execução permanente como administrador. Tela cheia e abertura automática são opções visíveis, não um sequestro do computador.

1.3 Autonomia sem pedir autorização a cada clique

Dentro de um objetivo e de um mandato válidos, pode planejar, ler materiais autorizados, pesquisar fontes públicas, calcular, criar rascunhos, gerar arquivos, testar código isolado e entregar ao canal verificado do proprietário. Não perguntar sobre decisões reversíveis sem importância que consiga resolver por contexto.

Pedir decisão quando houver mudança material de escopo, gasto não coberto, compartilhamento não autorizado, termos/obrigações novos, destruição irreversível, credencial necessária ou ambiguidade que altere a decisão. Pedidos devem ser concretos: ação, motivo, alternativas, dados, custo, renovação e consequência da recusa.

Autonomia não significa onisciência, acerto garantido, acesso a qualquer site ou direito de contornar controles. Se faltarem ferramenta, acesso ou evidência, tentar alternativas legítimas dentro do orçamento; depois explicar a limitação e preservar o trabalho.

1.4 O que não integra a primeira V1 completa

Avatar humano fotorrealista, clonagem de voz, controle do desktop pessoal, WhatsApp não oficial, cartões bancários irrestritos, investimento/trading autônomo, treinamento dos pesos do modelo, nuvem executora permanente, Windows, Mac Intel e vários funcionários comerciais simultâneos não são critérios da V1. APIs para extensões futuras devem permanecer possíveis sem implementar um SaaS complexo agora.

WhatsApp oficial é uma integração desejada e condicional a elegibilidade, conta, número, termos e custos verificados. O canal remoto obrigatório da V1 será um companion web autenticado para celular, com entrega por e-mail operacional quando configurada. Isso atende à comunicação remota sem fingir que WhatsApp está pronto. Não retirar a integração desejada do roadmap.

2. Definição de produto concluído

2.1 Três entregas diferentes

Alpha local confiável: conversa, memória, documentos, planejamento, execução de ferramentas confiáveis, controles, artefatos e recuperação funcionando com escopo limitado declarado. Não equivale ao funcionário completo.

Beta V1 integrada: acrescenta ambiente virtual real, navegador, pesquisa, conta operacional utilizável, voz, companion remoto, habilidades extensíveis controladas e instalação gráfica. Cada função essencial deve estar integrada, não apenas existir num módulo isolado.

Release V1: o mesmo produto é validado em Mac compatível, com segurança revisada, instalação em máquina limpa, assinatura/notarização, atualização, backup/restauração, exclusão, acessibilidade e operação cotidiana sem terminal. A publicação depende de aprovação humana informada pelas evidências.

2.2 O que não vale como comprovação

Uma pasta, classe, botão, screenshot, hash ou teste com modelo falso não comprova sozinho a capacidade correspondente. JSON válido não prova qualidade. Processo vivo não prova executor saudável. Arquivo íntegro não prova tarefa concluída. Conta criada não prova canal configurado. Uma captura da primeira janela não prova a jornada completa.

Mocks são obrigatórios nos testes determinísticos e proibidos como substituto silencioso de produção. Um teste ignorado deve aparecer como não executado, com causa. Nenhum gate pode ficar verde porque um teste crítico virou skip, xfail, continue-on-error ou perdeu suas asserções.

3. Arquitetura de referência e fronteiras reais

3.1 Stack e evolução sem recomeçar

Preservar Swift/SwiftUI no macOS, Python no núcleo, SQLite local com migrações e FTS5, contratos JSON Schema e IPC tipado. Não introduzir Redis, Kubernetes ou PostgreSQL na instalação pessoal para resolver problemas que cabem em transações locais. TypeScript pode ser usado no companion e no relay; eles não se tornam outro núcleo com autoridade.

O aplicativo macOS existente está em `platform/macos/AtlasKit`. Não recriar a interface em `apps/macos` apenas para corresponder a uma árvore antiga. Atualizar os índices e a documentação para apontarem para o código real. Preservar implementações aproveitáveis de Keychain, transporte, banco, broker e testes.

Fixar versões realmente compatíveis e verificadas no ambiente. Não copiar números de versão de uma resposta de IA sem resolver dependências, licença, origem, integridade e suporte ao alvo. Escrever ADR para toda mudança de fronteira de confiança.

3.2 Processos e responsabilidades

- **Atlas App:** interface, anexos escolhidos, microfone autorizado, prévias, mensagens e aprovações. Não possui shell genérico nem decide a política.
- **Atlas Supervisor:** helper Swift de usuário, vida dos serviços, exclusão de instâncias, reconexão, saúde e encerramento. Deve existir independentemente da janela; não ser apenas um objeto na memória da UI para a V1.

- **Atlas Core / Control Plane:** único escritor lógico do estado autoritativo. Requisições, tarefas, políticas, orçamento, aprovações, credenciais por referência, despacho e eventos.
- **Atlas Runtime:** interpretação, contexto, plano e propostas. Não aprova os próprios pedidos, não manipula diretamente segredos nem recebe capacidade genérica de escrever políticas.
- **Workspace Host:** helper Swift que mantém a VM, limites, canais guest/host e observação.
- **Guest Agent e ferramentas:** executam operações permitidas no ambiente do Atlas. Não recebem socket de autoridade, banco do Core, token de proprietário nem pastas pessoais.
- **Execution Box:** ambiente descartável separado para código novo, com dados mínimos, quotas e rede negada por padrão.
- **Gateway:** entrega remota, pareamento e filas. Relay não significa executor em nuvem.

Nomes de classes, processos separados sob o mesmo usuário ou verificações de `actor.kind` não constituem isolamento suficiente contra código arbitrário. A barreira de V1 depende de manter código não confiável fora do host, não compartilhar o banco e autenticar os canais. Um host já comprometido não está dentro da garantia de isolamento oferecida.

3.3 Fluxo obrigatório

```
Interface local / dispositivo pareado
-> recebimento durável e autenticação
-> intenção + contexto autorizado + tarefa versionada
-> proposta do Runtime
-> validação de contrato e do estado vigente
-> Policy + consentimento + orçamento + aprovação aplicável
-> registro da intenção de efeito
-> ferramenta confiável ou workspace autorizado
-> observação persistida + evidência + verificador
-> mudança de estado e outbox na mesma transação
-> resultado entregue e recibo registrado
```

O caminho de parada é independente dessa fila de trabalho. O modelo nunca é o portão obrigatório para que o proprietário consiga parar.

4. Contratos, esquema de dados e migrações

4.1 Contratos fechados e validação em todas as fronteiras

Validar o objeto completo antes de acessar campos, inclusive respostas do provedor, resultados de ferramentas e pedidos vindos da UI. Usar tipos fechados e discriminação por operação; rejeitar campos não previstos, valores negativos, moeda ausente, referências fora do escopo e payloads grandes. A saída estruturada do provedor é uma ajuda, não substituto da validação local. [R04]

Todo envelope inclui `schema_version`, `request_id`, `correlation_id`, `owner/employee` scope derivado da sessão, `issued_at` e versão de recurso quando aplicável. Não aceitar papel de proprietário fornecido pelo LLM no corpo. Comandos não idempotentes carregam chave estável e hash integral de parâmetros.

Adotar JSON-RPC com framing de tamanho e limite de controle de 1 MiB, preservando a negociação atual. Áudio, arquivos e tela usam canais próprios com quota e referências. Gerar tipos Swift/Python a partir da fonte de schema ou testar paridade automaticamente. Dinheiro usa moeda explícita; cálculo interno de inferência usa Decimal/unidade subcentavo para não acumular arredondamento por chamada. Converter para unidades mínimas somente na apresentação ou interface financeira que o exija.

4.2 Entidades adicionais e invariantes

Manter as tabelas existentes e acrescentar, por migrações compatíveis, pelo menos estas responsabilidades; nomes finais podem variar com ADR:

request_receipts: chave por empregado/operação, hash imutável, estado, vínculo com mensagem/tarefa, resultado ou erro, lease de processamento, timestamps. Restrição única sobre a chave; mesmo ID e payload diferente é conflito.

task_instruction_versions: pedido original integral, objetivo estruturado, restrições, anexos, critérios, revisão, autor autenticado, motivo de alteração. Tarefa aponta a revisão vigente. Resumo do modelo não substitui o pedido original.

task_controls: ordem de parada/pausa/cancelamento, escopo, epoch, origem, confirmação de aplicação, efeitos já despachados. **executor_leases:** dono, fencing token, heartbeat e expiração.

task_questions: pergunta com ID, tarefa, revisão, status, resposta e condição de retomada.

context_items / source_lineage: classificação, confiança, finalidade, fonte, derivação, validade, revogação e versão de conteúdo. **forget_tombstones:** escopo e origem esquecidos.

document_segments: artefato, versão, tipo de bloco, página/célula/slide, posição, texto/visual, estado de extração.

notification_outbox: evento único, destinatário/canal autorizado, mensagem ou referência de artefato, tentativas, expiração e recibo. **upload_sessions / import_receipts:** partes, tamanho, hash esperado, versão, quota, estado final e artifact_id recuperável.

model_validations: modelo, endpoint, versão da credencial por referência, capacidades, data, resultado e expiração. **price_versions:** fonte, moeda, componentes, vigência, verificação e tolerância. **workspace_sessions / browser_leases / skill_versions:** capacidades e vida operacional.

Foreign keys e transações precisam garantir: tarefa executável com entrada completa; ação ligada à tarefa/revisão; aprovação ligada à intenção exata; entrega com artefato real; evidência com critério específico; sessões e referências isoladas por proprietário. Não guardar senha, cookie ou chave bruta em nenhuma dessas tabelas.

4.3 Migração segura

Não editar SQL de migrações já aplicadas. Criar nova migração sequencial com hash, pré-condições, transformação, validação e compatibilidade de leitor. Testar banco vazio, banco Alpha 1, banco Alpha 2, migração interrompida e tentativa de abrir banco mais novo com binário antigo.

Antes de alteração destrutiva, obter backup consistente e verificável; se o formato já for cifrado, preservar isso. Preservar IDs de identidade, conversas, tarefas e arquivos. Não apagar o banco para fazer a atualização passar. Se a migração não puder recuperar automaticamente uma condição, gerar diagnóstico e procedimento de restauração sem efeitos externos.

5. Recebimento durável, idempotência e eventos

5.1 Envelope imutável de pedido

O cliente persiste antes do envio: texto integral, anexos com hashes, intenção explícita quando houver, pergunta/tarefa de destino, revisões e chave idempotente. Reenviar é repetir exatamente esse envelope. Editar qualquer campo cria nova revisão ou novo pedido; nunca reutilizar a mesma chave com outro conteúdo.

No Core, uma transação registra receipt e mensagem. O processamento pode ocorrer depois por worker supervisionado. Estados mínimos do receipt: RECEIVED, PROCESSING, COMPLETED, FAILED e RECONCILIATION_REQUIRED. Queda após persistir uma mensagem não pode terminar permanentemente em duplicate com resultado vazio.

Uma consulta pelo ID deve devolver resultado final ou estado real de processamento. Se o processo cair depois de uma inferência cobrada, reconciliar essa tentativa antes de decidir uma nova chamada. Não prometer execução exatamente uma vez em qualquer serviço externo: garantir deduplicação local e combinar idempotência do provedor com reconciliação.

5.2 Publicação da tarefa

Persistir objetivo integral, critérios, política de dados, conversa e todos os anexos numa única transação antes de tornar a tarefa elegível. Alternativa válida: estado STAGING inelegível, com transição única para CREATED/READY depois da validação completa. O worker não pode observar um pedido parcialmente preparado.

Não manter transação SQLite aberta durante chamadas de rede ou inferência. Separar preparação, despacho e liquidação. Proteger concorrência por restrições e operação transacional; locks somente em objetos Python/Swift não coordenam processos distintos.

5.3 Outbox e entrega

Mudança para aguardando usuário, aprovação, bloqueio ou conclusão persiste a notificação na mesma transação. Um dispatcher separado entrega e registra recibo; falha preserva a pendência. Deduplicação por event_id + channel + recipient impede múltiplos balões locais.

Em serviços externos sem idempotência, a entrega pode ser desconhecida. Antes de reenviar, consultar recibo ou conciliar. Não prometer exatamente uma vez por e-mail/serviço que não forneça essa propriedade. O canal deve distinguir recebido pelo Atlas, enviado, aceito pelo provedor e lido, quando tal confirmação realmente existir.

6. Controle imediato, tarefas e recuperação

6.1 Máquina de estados

Preservar CREATED, UNDERSTANDING, PLANNING, READY, RUNNING, WAITING_USER, WAITING_APPROVAL, BLOCKED, RETRYING, PAUSED, VERIFYING, COMPLETED, FAILED e CANCELLED. STAGING pode ser acrescentado por migração para preparo atômico. Entrega parcial é resultado explícito; não usar COMPLETED sem cumprir critérios obrigatórios ou registrar redução de escopo autorizada.

O Core retorna available_actions e blocked_reason para cada estado. UI e companion não adivinham botões. “Retomar” de PAUSED é diferente de “Tentar novamente” após indisponibilidade e “Reavaliar bloqueio” após mudar orçamento. UNKNOWN externo nunca vira READY somente porque um botão foi apertado.

6.2 Canal de controle prioritário

Implementar control.stop, tasks.pause, tasks.cancel e tasks.update_instruction fora da fila serial de inferência/chat. A autenticação é a mesma autoridade, mas o socket/fila de controle não fica atrás de upload ou modelo lento. Disponibilizar um botão visível de parada e atalho; isSending não pode descartá-los.

Parar deve persistir intenção, elevar control_epoch, impedir novos grants, revogar leases de despacho e solicitar cancelamento das operações interrompíveis. O broker confere epoch/revisão

imediatamente antes do envio. A confirmação explica o que parou, o que foi enviado e o que exige reconciliação.

Definir o ponto de linearização: uma operação já autorizada e enviada pode terminar depois da parada. Não apagar seu histórico ou afirmar que foi desfeita. Nenhuma operação que começa a ser autorizada depois do commit da parada pode despachar com epoch antigo. Metas de 1 s para reconhecimento local e 2 s para barrar novos despachos valem no Mac de referência sob a carga definida nos testes.

6.3 Correções durante o trabalho

Associar a correção à tarefa correta, preferindo referência explícita quando houver ambiguidade. Persistir nova `instruction_version` e classificar quais propostas, critérios e aprovações ficaram obsoletos. Conferir a versão antes de cada inferência e de cada efeito. Replanejar apenas a parte necessária; preservar as ações já realizadas.

Uma confirmação “vou considerar” só pode ser emitida depois do registro durável. Se a ação material já ocorreu, responder isso claramente. Uma correção não concede capacidade, aumenta orçamento nem confirma compra por inferência semântica.

6.4 Heartbeat, watchdog e scheduler

Heartbeat não pode depender da volta de uma chamada bloqueante. Utilizar tarefa supervisora com conexão de banco própria, revogação e fencing. TTL inicial de 60 s e heartbeat até 15 s são propostas calibráveis; não corrigir falhas apenas aumentando TTL. O watchdog verifica executor ausente, lease expirado e falta de progresso separadamente.

O scheduler roda independentemente do worker que executa tarefas longas. Usa UTC para instantes e IANA para horários civis. Jobs carregam recorrência, timezone, política de execução perdida, tolerância de atraso e chave de ocorrência. No retorno após offline, consolidar ou pular ocorrências conforme a política; não disparar várias ações externas acumuladas sem revisar impacto.

Manter limites por tentativa, tempo, custo e ausência de progresso. Três tentativas transitórias e dois replanejamentos sem progresso são defaults iniciais; vinte passos é limite de ausência de progresso, não proibição de uma tarefa longa produtiva. Circuit breakers persistem ou são compartilhados entre clientes do mesmo provedor para não zerar a cada criação de objeto.

6.5 Falhas e saúde

Capturar respostas malformadas antes de indexação/slicing. Tratar erro inesperado do worker como evento visível, invalidar seu lease e recuperar a tarefa com reconciliação. Não usar `except Exception: pass` para fingir que a operação terminou. Permitir retomada controlada, sem repetição de efeito incerto.

Saúde inclui: processo, worker vivo, heartbeat, última atividade, filas, leases vencidos, banco, Vault, modelo/credencial, orçamento, rede, VM, canal remoto e dispatcher. “Conectado” é somente estado de conexão; não deve se confundir com “apto a executar este pedido”.

7. Conversa natural, continuidade e identidade

7.1 Intenções que precisam funcionar

A conversa deve distinguir `CHAT`, `DELEGATE`, `TASK_STATUS`, `ANSWER_QUESTION`, `CORRECT_TASK`, `REMEMBER`, `SHARE_ONLY`, `ATTACH_TO_TASK`, `SCHEDULE`, `PAUSE`, `STOP`, `CANCEL` e `APPROVAL_RESPONSE`. Os nomes são contratos propostos, não permissões concedidas pelo classificador. Saudação não vira tarefa; anexo não implica delegação; mensagem nova não é automaticamente resposta à única pergunta pendente.

Usar parser determinístico estrito para controles exatos e eventos de botões. Para linguagem geral, usar contexto e classificador estruturado com possibilidade de ambiguidade. “Como está o tempo?” não é status de tarefa; “lembrete” não deve ser cortado como se fosse “lembre”. Preservar a frase original mesmo ao normalizar acentos para reconhecimento.

Conversa sem tarefa também pode buscar memória e consultar estado através de ferramentas de leitura escopadas. Ela não deve ficar presa às últimas vinte mensagens nem criar uma tarefa artificial só para lembrar uma preferência. Perguntas de informação atual exigem fonte atual; o modelo não pode simular pesquisa.

7.2 Vínculos explícitos sem interface complicada

Toda pergunta do Atlas tem `question_id`, tarefa e revisão. Oferecer “Responder a esta pergunta” e um indicador discreto da tarefa no campo de composição. Com continuidade inequívoca, vincular automaticamente; com duas perguntas plausíveis, perguntar qual delas, mostrando títulos curtos. Pedidos de status ou novos assuntos não podem ser consumidos como resposta pendente.

Ao mencionar “aquele relatório”, recuperar contexto e candidatos; se houver um único correspondente, continuar. Quando existirem várias opções relevantes, mostrar escolhas, sem exigir IDs técnicos. Uma instrução “esse é o arquivo que faltava” anexa ao trabalho alvo; “guarde isso” armazena; “compare com este outro” delega ou altera a tarefa apropriada.

Manter um histórico contínuo por funcionário com threads/tarefas relacionadas, sem exigir que o proprietário organize projetos antes de falar. Títulos, marcação de resultado, anexos e status devem ajudar, não transformar a conversa numa tela de depuração.

7.3 Contexto conversacional e qualidade das respostas

Incluir identidade configurada, idioma, pedido vigente, restrições ativas, memórias relevantes, estado solicitado e histórico selecionado. Reter papéis de usuário e assistente. Resposta antiga do LLM não vira `VERIFIED_FACT` automaticamente. Conteúdo de documento, citação e resultado de ferramenta mantém origem externa mesmo depois de resumido.

Preservar o pedido atual antes do histórico. Reservar espaço para controles e restrições; se não couberem, dividir ou pedir esclarecimento, não omitir silenciosamente. Compaction produz resumo com fontes e versão, não substitui o acervo canônico. Reconstruir contexto após reinício e após exclusões.

Responder com conteúdo efetivo. Quando aceitar trabalho, apresentar objetivo e condições essenciais a partir da tarefa persistida. Progresso vem dos eventos; não inventar porcentagens ou dizer “estou pesquisando” quando nenhuma ferramenta de pesquisa foi usada. Não despejar raciocínio privado do modelo: fornecer decisões e justificativas operacionais resumidas.

7.4 Conversa durante outras tarefas

O usuário deve poder conversar, fazer perguntas, corrigir e interromper enquanto o Atlas trabalha. Não bloquear chat por uma tarefa longa. Separar processamento interativo, trabalho e controle. Aplicar orçamento comum, mas prioridade maior à interação humana e caminho imediato à parada.

Uma pergunta simples não deve custar uma cadeia longa de agentes. Quando um pedido de trabalho depende da resposta a outra conversa, registrar dependência. Não esquecer um objetivo só porque chegaram mensagens mais recentes.

8. Memória duradoura, temporal e aprendizado

8.1 Tipos e estados

Manter identidade, preferências, fatos, episódios, procedimentos e memória operacional. Cada item carrega proprietário/funcionário, versão, conteúdo ou referência, fonte, classificação, finalidade, confiança, vigência, data de registro, confirmação e estado. Segredos e permissões não são memória semântica.

Estados mínimos: proposed, confirmed, disputed, superseded e deleted. Confiança é uma categoria operacional com justificativa; não apresentar porcentagem de certeza sem calibração. Dados confirmados pelo proprietário não são automaticamente verdade universal: distinguir “informado pelo proprietário” de “verificado por fonte externa”.

8.2 Recuperação comum a chat e tarefas

Implementar KnowledgeContextService ou responsabilidade equivalente. Fluxo: determinar assunto e finalidade; filtrar por proprietário, acesso, tombstone e vigência; combinar busca textual e semântica/paráfrases; ordenar por pertinência e atualidade; recuperar trechos com fonte; montar somente o contexto necessário.

Identidade e preferências globais aplicáveis não devem depender de coincidência de palavras na pergunta. Índice semântico é derivado e reconstruível. Embeddings locais são preferíveis para conteúdo sensível; embeddings remotos são uma divulgação de dados e exigem a mesma autorização de saída que uma chamada ao modelo. Não rebaixar classificação para permitir indexação.

Meta de aceite: guardar fato sintético, inserir cem mensagens sobre outros assuntos, reiniciar, perguntar por paráfrase e recuperar a fonte correta. Testar também busca sem resultado e conflito de versões. Não preencher lacunas inventando lembranças.

8.3 Atualização e tempo

Corrigir conteúdo preserva valid_from, valid_until, timezone e fonte anterior quando esses campos não forem explicitamente alterados. Diferenciar data de registro da data de validade. Em conflito, manter versões, status e proveniência, escolhendo a vigente para uso operacional.

Agenda e datas usam estruturas, não apenas texto vetorial. Eventos de dia inteiro, fuso, recorrências, exceções, intervalos e horário de verão precisam de testes. Consultas temporais devem recuperar os eventos relevantes e calcular sobreposição por código determinístico. O exemplo familiar não deve ser hard-coded: o mesmo motor atende qualquer conjunto autorizado de compromissos.

8.4 Esquecimento operacional e exclusão

Oferecer tela Memória com fonte, conteúdo, vigência, confirmação, editar, excluir e exportar. “Não use mais isto” cria bloqueio de recuperação; “apague o conteúdo sob seu controle” inclui fontes/derivações selecionadas, histórico aplicável, índices, caches e observações, com prévia do alcance.

Manter tombstones por origem/derivação para evitar que uma confirmação antiga ou resumo reintroduza o dado. A auditoria operacional pode reter IDs e motivo sem reter o conteúdo removido. Backups possuem retenção e regras de restauração que reapliquem tombstones. Não prometer apagar cópias de terceiros, dados fora do controle do produto ou setores físicos de SSD.

8.5 Aprender com feedback e com experiência

Feedback como “mais curto”, “priorize prazo” ou “isso funcionou” gera preferência proposta ou lição ligada a tarefa, evidência e condições de validade. Aprendizado de procedimento não altera pesos do LLM nem concede poderes. Não reexecutar tarefas externas por conta própria para “treinar”.

Após conclusão, consolidar episódio: objetivo, método utilizado, falha, correção, resultado e teste de reutilização. Consolidar em baixa prioridade com orçamento próprio. Uma lição não verificada não é regra global. Skills aprovadas podem ser reutilizadas conforme o contrato do capítulo 17.

9. Privacidade, classificação e saída de dados

9.1 Dois eixos independentes

Não confundir **confiança/autoridade** com **sensibilidade**. Um documento externo pode ser altamente sensível e não confiável como instrução. Uma instrução autenticada pode conter dados que não podem ir a um provedor. Preservar os dois eixos em toda transformação.

Classes operacionais: PUBLIC, INTERNAL, PERSONAL, SENSITIVE e SECRET, com hierarquia e exceções formalizadas. `unknown classification` em conteúdo importado não equivale a PUBLIC; manter em quarentena/restrito até classificação. Esses rótulos são decisões técnicas, não definições jurídicas universais.

Um resultado derivado herda pelo menos a proteção dos dados utilizados, salvo desidentificação validada e política específica. Resumos, OCR, embeddings, transcrições, nomes de arquivo, screenshots, logs e saída de ferramenta entram nessa regra.

9.2 Verificação obrigatória antes da rede

O Egress Guard considera o payload final, proveniência, classificação agregada, finalidade, destinatário/provedor, escopo do consentimento e revisão vigente. Deve atuar fora do texto do LLM, imediatamente antes do transporte. Uma autorização de armazenamento não é autorização para envio.

Sem consentimento compatível, bloquear a divulgação e oferecer alternativa: processamento local, retirar um trecho, pedir autorização escopada ou produzir entrega parcial. Não mandar o mesmo dado para outro modelo como fallback. SECRET nunca entra no contexto nem em embeddings; o cofre libera seu uso apenas ao adaptador de autenticação destinado ao serviço autorizado.

Teste sentinelas: plantar conteúdo sintético em cada via de entrada e inspecionar o payload efetivamente encaminhado a servidores locais. Nenhuma sentinela proibida pode aparecer após resumo, confirmação, correção, reinício, exportação ou troca de provedor.

9.3 Retenção e controles do usuário

Áudio bruto não é guardado por padrão depois do processamento. Screenshots só ficam quando necessários à evidência e segundo prazo configurado. Logs técnicos redigidos: trinta dias como default proposto; conversas e entregáveis: política visível e ajustável. Exportar dados com manifesto e referências; excluir com alcance explicado.

O usuário deve saber quais processamentos dependem da nuvem. `store=false` não será apresentado como promessa universal de retenção zero do fornecedor; conferir as políticas, opções e condições da conta na documentação aplicável. [R05] Conformidade jurídica depende da operação e de revisão própria; este contrato não certifica LGPD.

10. Documentos: compreender, não apenas receber arquivos

10.1 Pipeline de ingestão

Receber cópia explícita, calcular hash, verificar tamanho/tipo, colocar em staging, validar arquivo e extrair em processo limitado ou Execution Box. Nunca executar macros, JavaScript, links externos de planilhas ou código embutido para “ler”. Preservar o original como artefato imutável.

Estados por arquivo: RECEIVING, QUARANTINED, STORED, EXTRACTING, READY_FOR_ANALYSIS, PARTIAL, UNSUPPORTED e FAILED. Mostrar “armazenado, mas ainda não analisável” quando esse for o caso. Gerar diagnóstico por arquivo, não resposta genérica de que todos os anexos foram entendidos.

10.2 Extratores exigidos para V1

TXT/Markdown/CSV/JSON: decodificação com detecção explícita de encoding quando necessário, estrutura, delimitador e erros reportados. CSV com números locais precisa preservar valor bruto e interpretação, sem converter silenciosamente moeda ou decimal.

PDF com texto: recuperar página, texto, tabelas/posições quando necessário e ordem de leitura; renderizar trechos com layout ambíguo. **PDF digitalizado/imagem:** usar visão/OCR apenas quando o texto não estiver disponível ou for insuficiente, com autorização de processamento e avaliação de qualidade. Erro de OCR não pode virar número confiável sem verificação.

DOCX: extrair parágrafos, tabelas, cabeçalhos/rodapés e notas relevantes; tratar revisões/comentários conforme modo declarado. **XLSX:** planilha, célula, fórmula, valor armazenado e formatação de unidade; conferir células ocultas e links externos sem executá-los. Não afirmar recálculo se não houver motor validado. **PPTX:** slides, texto, tabelas, notas e imagens relevantes, com referência ao slide.

Bibliotecas candidatas são pypdf, python-docx, openpyxl e python-pptx, complementadas por renderização/visão quando necessário. A escolha final exige documentação, licenças, suporte ao pacote e testes de fidelidade; não assumir que uma biblioteca simples extrai todas as tabelas e desenhos corretamente. Não exigir aplicações de escritório do usuário para o uso normal.

10.3 Contrato de leitura por trechos

`documents.read` retorna `document_id`, `artifact_version`, `segment_ids`, conteúdo tipado, localizadores, tamanho total, `next_cursor`, `has_more`, `coverage` e avisos. `documents.search` localiza trechos; `documents.inspect_visual` lê a evidência visual autorizada. O conteúdo completo permanece acessível, mesmo quando uma chamada só recebe um resumo.

Não cortar JSON serializado para caber em coluna ou contexto. Persistir observação válida com `data_refs` para o conteúdo maior. Cada segmento deve caber no limite do contrato; textos enormes são divididos antes da serialização. Trechos decisivos após vinte mil e duzentos mil caracteres precisam ser recuperáveis.

10.4 Cobertura e comprovação

Registrar quais páginas/planilhas/segmentos foram processados e quais foram consultados pela tarefa. Busca focal não deve ser anunciada como leitura completa. Para “revise o documento inteiro”, planejar cobertura integral ou explicar partes não lidas. Tabelas, cláusulas finais e exceções precisam integrar os testes, não apenas o primeiro parágrafo.

Construir fixtures com o mesmo conteúdo em diferentes formatos, campos numéricos e uma cláusula decisiva no final. Comparar extração com gabarito. Fonte cifrada sem senha, arquivo corrompido ou layout irresolúvel deve gerar intervenção específica, sem inventar análise.

11. Modelos de IA e configuração de inteligência

11.1 Escolha inicial e validação

O catálogo oficial em inglês consultado em 24/09/2026 identifica `gpt-6-sol`, `gpt-6-astra` e `gpt-6-luna`. São candidatos iniciais, não garantia de acesso em determinada conta. Adotar Sol para trabalho geral, Astra para revisão/raciocínio difícil e Luna para tarefas leves somente após avaliações do Atlas. [R01-R03]

Para a construção e uma referência inicial de qualidade, usar o modelo de programação mais capaz disponível no ambiente autorizado. No produto, manter o provedor desacoplado. Começar homologando um provedor de ponta a ponta, sem exigir duas assinaturas do usuário; adicionar outro só com consentimento e equivalência de contratos.

Não derivar IDs de apelidos ou nomes do chat. O catálogo da aplicação guarda ID real, versão/snapshot quando existente, capacidades, preços, datas de verificação e acesso da conta. Atualizações do catálogo não substituem automaticamente um modelo em tarefas de alto impacto sem avaliação.

11.2 Perfis e parâmetros reais

Automático: filtrar primeiro privacidade, capacidade, disponibilidade e orçamento; depois escolher custo/latência/qualidade avaliados. **Econômico:** menor custo entre os perfis que passam nos testes pertinentes. **Máxima qualidade:** perfil mais forte habilitado, sem elevar orçamento. **Manual:** modelo selecionado explicitamente. Nenhum botão pode alterar apenas um rótulo sem mudar o comportamento.

Parâmetros avançados: esforço suportado, limite de saída, prazo por operação, orçamento de contexto, fallback, paralelismo e ferramentas permitidas. Omitir temperatura ou parâmetros não suportados; não mapear níveis de raciocínio de provedores diferentes como se fossem equivalentes. Começar com esforço intermediário e avaliar; “max” em todas as mensagens não é requisito de inteligência.

Estabelecer baseline comparável: mesmos pedidos sintéticos, mesmas fontes, avaliações e orçamento; comparar Sol/Astra antes de baixar qualidade por economia. Router otimiza custo por tarefa concluída corretamente, não apenas preço por token. Classificador leve não libera pagamentos ou outras autorizações.

11.3 Contrato do provedor

Manter generate, stream, cancel, estimate/count usage, health e capabilities. Normalizar resposta, request ID, modelo efetivo, motivo de término, uso reportado e erro. Erro de credencial pede reautorização; recusa de política não é contornada; indisponibilidade usa backoff com limite. Validar JSON, tipos e schema completo no consumidor. [R04]

A validação de acesso deve estar vinculada a `credential_ref` + `credential_version` + `endpoint` + `model_id` + `capability_version`. Trocar chave/conta invalida a indicação Pronta. Separar estado de acesso, compatibilidade, preços e avaliação funcional. Um pequeno JSON de saúde não homologa autonomia.

Contexto não deve preservar instruções antigas e eliminar a mensagem atual. Registrar por IDs o que entrou/saiu e por quê, sem despejar dados sensíveis nos logs. Respostas parciais de streaming, interrupção e limites de tokens devem ser casos explícitos de retorno.

11.4 Configuração para pessoa comum

Na tela principal: “Automático”, “Econômico” e “Máxima qualidade”, com estimativa de gasto e explicação. Catálogo deve listar apenas modelos compatíveis e dizer quais faltam validar. ID técnico fica em Avançado. Chave inserida em campo seguro vai ao Keychain; nunca é repetida no chat.

Oferecer onboarding BYOK guiado na Alpha. Para a experiência comercial sem chave, implementar modo gerenciado real com serviço de inferência autenticado, cotas, cobrança e consentimento de nuvem próprios; não embutir uma chave-mestra de fornecedor no aplicativo. A V1 pode manter BYOK guiado como modo operacional, mas não chamar esse modo de “sem configuração de conta”. API e assinatura ChatGPT possuem faturamento separado. [R06]

12. Orçamento, credenciais e autorizações

12.1 Orçamento vigente em cada ação

Separar inferência, ferramentas pagas e compras. Nulos significam não configurado, nunca ilimitado. Ler a revisão vigente de limite e consentimento dentro da transação de cada nova reserva. Cliente criado antes de uma redução não pode continuar autorizando gastos pelo limite antigo.

Exibir usado, reservado, desconhecido, estimado e restante. Redução abaixo do já gasto não apaga consumo; impede novas despesas. Reserva de ação já enviada pode permanecer até conciliação. Alertas de setenta e noventa por cento são informativos e não substituem bloqueio de novas autorizações.

Usar tabela versionada com fonte e vigência. Preço de referência não verificado exige aceite como estimativa; nunca chamar o valor resultante de teto universal. Tokens devem ser contados com mecanismo documentado ou estimativa conservadora validada, incluindo texto, imagens, áudio, ferramentas e overhead aplicáveis. `caracteres/2` não é limite universal.

12.2 Tentativas e cobrança desconhecida

Registrar tentativa e reserva antes do envio. Sem envio/cobrança comprovados: liberar. Uso confirmado: liquidar. Resultado incerto: manter UNKNOWN com reconciliação. Se o provedor não fornecer uso, registrar estimativa conservadora e distingui-la do valor faturado. Não liberar tudo num finally.

Teste de inteligência, chat, busca semântica remota, revisão e voz entram no mesmo registro de consumo, categorizados. O teste de inteligência tem teto por execução e também respeita o período global. Controle de deduplicação impede que reconexão ou clique duplo disparem múltiplos testes pagos.

12.3 Mandatos e aprovação exata

Mandato autoriza uma classe delimitada de ações por finalidade, destinatários, dados, número de usos, prazo e teto. Exemplo: enviar entregáveis não sensíveis ao endereço verificado do proprietário sem perguntar em todo envio. Não aceitar “faça o que quiser” como autorização para dinheiro, terceiros, segredos ou administração.

Aprovação formal contém ação, tarefa/revisão, destino, fornecedor, dados/anexos, itens, valor máximo/moeda, taxas conhecidas, recorrência, termos materiais, validade e nonce. Alteração material invalida a aprovação. Reserva e consumo devem ser atômicos com o despacho; replay não funciona.

R0/R1: leitura autorizada e criação local reversível; R2: entrega no canal do proprietário já consentida; R3: contato com terceiros/cadastro sob mandato ou aprovação; R4: compra, assinatura, cancelamento oneroso ou exclusão definitiva exige aprovação forte; R5: proibição. Essas classes não são uma média: uma proibição não é compensada por pontuação baixa em outro item.

“Pode” pode confirmar somente uma solicitação única e recente de baixo risco dentro de sessão autenticada. Operação financeira abre cartão de confirmação forte, por exemplo confirmação local com mecanismo nativo validado. Frase numa página, áudio ambiente ou e-mail não vale como autorização do proprietário.

12.4 Uso de credenciais

Cofre nativo guarda segredos; banco guarda referências, escopos, versão, expiração e destino. O adaptador recebe o mínimo pelo tempo necessário. Sem `read_secret` para o LLM, dumps de cookies, segredos nos screenshots ou chaves em variáveis compartilhadas com código experimental.

Transporte autenticado recusa redirecionamentos por padrão. Exceção requer origem e finalidade revalidadas; nunca levar Authorization a outra origem. Fazer parsing de scheme/host/porta; HTTP somente para loopback exato em modo de testes. Endpoints normais são allowlist de distribuição; mudança é configuração de confiança, não decisão do modelo.

13. Planejamento, ferramentas e conclusão comprovada

13.1 Pedido integral e plano verificável

Preservar original, anexos e correções. Derivar objetivo, restrições duras, entregáveis, fontes necessárias, subtarefas, dependências e critérios. Pedir esclarecimento somente se faltar informação material. Hipóteses reversíveis de baixo risco podem ser adotadas e registradas.

Plano não é uma lista ornamental: cada passo tem entrada, ferramenta/capacidade, resultado esperado, verificação e condição de falha. Replanejar com razões operacionais. Máximo inicial de dois pesquisadores independentes e um executor de efeitos externos por workspace; não criar agentes ilimitados.

13.2 Ferramentas de produção

Cada manifesto inclui ID/versão, schema de entrada/saída, efeito, capacidades, classificação esperada, limites de rede, prazo, idempotência, recibo e verificador. Começa desabilitado até homologação. Catálogo mostrado ao modelo inclui somente ferramentas habilitadas para aquele contexto. Risco/efeito vem do manifesto confiável, não da autodescrição do LLM.

Catálogo mínimo de V1: busca/leitura de conhecimento; leitura paginada de documentos; cálculo e datas; pesquisa pública; browser no guest; geração e validação de artefatos; contatos/contas operacionais autorizadas; envio ao proprietário; agendamento; execução de código em sandbox; proposta/teste de skill. Capacidades não implementadas devem ser visíveis como ausentes, não inventadas numa resposta.

13.3 Verificação por critério

Separar: integridade do arquivo; abertura; cobertura do pedido; exatidão de cálculos; existência/proveniência das fontes; completude; política; entrega. Não satisfazer todos os critérios com uma evidência genérica “arquivo abre”.

Pesquisa resolve as fontes para registros realmente recuperados, com URL, data e trecho de suporte. Uma string parecida com URL não basta. Cálculos usam código e unidades explícitas; planilhas exigem validação de fórmulas/valores. Relatórios devem comparar as dimensões solicitadas. Critério subjetivo pode ter avaliador complementar e revisão humana; outro LLM não é prova de verdade factual.

Resultado final usa COMPLETED só quando os critérios obrigatórios foram atendidos. Senão, apresentar parcial com faltas e ação seguinte ou aguardar aprovação de alteração de escopo. O próprio usuário pode dispensar um critério, mas a mudança precisa ser registrada e não retroativamente inventada pelo agente.

Placeholder deve ser contextualizado: aceitar a palavra portuguesa “todo”; rejeitar marcadores como [inserir valor] em conteúdo que deveria estar pronto. Template de código que ensina TODO pode ser legítimo se esse for o objetivo; a regra não deve bloquear exemplos autorizados indiscriminadamente.

14. Arquivos, artefatos e qualidade visual

14.1 Upload e importação

Fila com item por arquivo, progresso, cancelamento e erro específico; arrastar dez arquivos não pode descartar nove por isAttaching. Mesma implementação para seletor e drag-and-drop. Staging possui quota individual e total, TTL, trava por upload entre conexões, hash esperado e finalização transacional.

Importação produz recibo consultável por ID. Se a resposta cair, recuperar artifact_id existente em vez de duplicar. Arquivo final só é publicado após validação completa. Temporários abandonados são removidos de forma recuperável sem apagar importação em andamento.

14.2 Leitura/exportação eficiente e segura

Artefatos são imutáveis por versão, armazenados por referência. Leitura por chunks usa sessão e tamanho/hash fixos; deve ser aproximadamente linear no tamanho total, não recalculando arquivo inteiro para cada trecho. Verificação final mantém a integridade e detecta mudança durante a leitura.

Salvar como sugere nome e extensão corretos; renomear não muda o formato sem conversão real. Usar escrita atômica, consentimento para sobrescrever, feedback e abertura pelo aplicativo apropriado. Prévia de HTML/código é inerte; conteúdo ativo não executa ao visualizar.

14.3 Formatos de entrega

V1 deve gerar pelo menos TXT/Markdown, PDF, DOCX, CSV/XLSX e apresentação PPTX quando a habilidade correspondente for usada. Geração executa em biblioteca/ferramenta instalada e validada, nunca devolve apenas um caminho imaginário. Arquivo final tem bytes, MIME, extensão, hash, fonte e versão.

PDF/documento/apresentação: renderizar, inspecionar layout e validar tabelas, páginas, caracteres e conteúdo solicitado. Planilha: fórmulas preservadas, tipos/unidades certos, sem erro conhecido, indicação de recálculo quando não comprovado. Código: testes e build no ambiente correspondente; não declarar software funcional só por existir arquivo-fonte.

Não prometer excelência universal em qualquer domínio: a habilidade deve declarar capacidades e limites. Quando exigir ferramenta externa, buscar alternativa permitida ou propor aquisição/autorização, preservando o objetivo do usuário.

15. O computador próprio do Atlas

15.1 Workspace virtualizado obrigatório para a visão completa

Implementar `WorkspaceProvider` com provision, start, health, pause, resume, stop, observe, takeover, execute_tool e transferência de artefatos. O provider de referência é VM Linux ARM64 no Mac Apple Silicon por `Virtualization.framework`. Esta é uma decisão do produto a comprovar no equipamento; não uma afirmação de que a VM já existe na Alpha.

Alvo inicial: macOS 15 em Apple Silicon, 16 GB como hipótese mínima de engenharia e 32 GB como preferência. Medir antes de publicar compatibilidade; equipamento exato do proprietário ainda precisa ser diagnosticado. Apps exclusivamente macOS não funcionam automaticamente dentro de um guest Linux; tratá-los como provider/habilidade futura ou alternativa permitida.

O setup baixa ou provisiona imagem versionada, verifica assinatura/hash e licença, mostra tamanho/progresso e retoma download. Configurar recursos, espaço livre, resize e limpeza com quotas. Falta de espaço ou falha de boot produz diagnóstico. Não substituir VM quebrada por execução no desktop pessoal.

15.2 Isolamento de disco, autoridade e código novo

Não montar home, iCloud, Keychain, banco do Core, tokens de proprietário ou repositório da instalação na VM. Importação/exportação usa broker por artefato. Clipboard, pastas compartilhadas e USB ficam desabilitados por padrão; permissões específicas precisam de decisão separada.

O workspace operacional mantém navegador e arquivos próprios. Código novo roda em Execution Box descartável, separada das sessões autenticadas. Um container pode ser uma defesa adicional dentro do guest, mas não deve ser apresentado como fronteira equivalente à VM contra código arbitrário privilegiado.

A V1 não mantém sessões bancárias ou credenciais financeiras de alto privilégio no browser geral. Conta de teste ou operacional também tem escopo: código experimental não herda seus cookies. Snapshot não autoriza reaplicar ações externas antigas; restauração exige reconciliar ledger e invalidar grants.

15.3 Transporte e saída de rede

Validar uma solução sem saída direta: guest sem NIC de rede geral e comunicação guest/host por canal virtio-socket autenticado; proxy local no guest encaminha somente pedidos permitidos a um mediador fora da VM. A Apple documenta o dispositivo virtio-socket para comunicação host/guest; política de proxy, autenticação, quotas e isolamento são responsabilidades a implementar no Atlas. [R07]

Se essa prova não atender aos navegadores necessários, avaliar `VZFileHandleNetworkDeviceAttachment` com biblioteca de rede em espaço de usuário ou outra arquitetura auditável. Registrar ADR e testes. Não escrever uma pilha TCP artesanal nem trocar por NAT irrestrito silenciosamente. Uma alternativa deve manter egress verificável fora do controle de código executado no guest. [R08]

O mediador bloqueia loopback do host, LAN, link-local, metadados, redes privadas, multicast e destinos fora do mandato. Valida IPv4/IPv6, DNS e revalidação de destino, portas e quotas; trata conexões TCP, WebRTC/UDP, DNS alternativo e túneis. A ausência de uma NIC geral reduz caminhos de bypass, mas não substitui os testes.

Serviços sensíveis usam grants vinculados à sessão, tarefa/revisão e destinos. Para pesquisa pública, permitir navegação mais ampla sem credenciais privilegiadas, respeitando política de acesso. Não afirmar que allowlist de domínios entende o conteúdo de todo formulário HTTPS: escrita de alto impacto precisa de API/driver homologado ou supervisão humana.

15.4 Prova exigida

Testar guest tentando ler arquivo-sentinela no host, socket de autoridade, tokens, rede local, endereço de metadados e egress não autorizado. Testar código experimental tentando ler cookies da sessão operacional. Todos os bloqueios críticos devem ter evidência real no Mac, não retornar `False` em um mock.

Fechar a janela não encerra VM autorizada. Suspensão e desligamento interrompem capacidade local; registrar isso corretamente. “Ver trabalhando” mostra frames reais do guest, com timestamp e atividade, nunca vídeo simulado.

16. Navegador, pesquisa e superação legítima de obstáculos

16.1 Ordem de escolha de ferramenta

Preferir API oficial e integração tipada quando oferecerem o dado ou ação necessária dentro do escopo. Depois usar Playwright com navegação estruturada no guest. Visão e ações sobre screenshot são fallback para interfaces que exigem isso; cada ação precisa de observação recente e validação. O modelo sugere o gesto; o broker o autoriza e executa.

Empacotar versões compatíveis de Playwright e seus navegadores, em perfil próprio. Estado autenticado inclui cookies e outros dados capazes de autorizar acesso; tratar como segredo, não publicar no Git ou em logs. [R09, R10]

16.2 Navegação geral não é autorização geral

Manter perfis separados para pesquisa pública, contas operacionais e operações assistidas. Limitar downloads, uploads, popup, esquemas de URL, redirecionamentos e navegação a arquivos locais. Links `file:`, chamadas ao host e protocolos executáveis não podem virar escape.

Para ação externa, o driver identifica semanticamente destinatário, dados, finalidade e custo. Cliques de compra/envio em site não homologado exigem takeover ou aprovação estruturada com garantia suficiente de que o clique corresponde à ação aprovada. Não usar classificações vagas do LLM como único controle.

16.3 Pesquisa com evidência

Registrar busca, termos, fonte, horário, documento recuperado e trechos que sustentam cada conclusão. Distinguir preço observado, disponibilidade confirmada e estimativa. Informações temporais devem ter data de consulta. Não inventar consulta a site bloqueado nem esconder divergência entre fontes.

A resposta deve ser útil mesmo quando a primeira fonte falha. Tentar API permitida, outra fonte pública, nova consulta ou solicitar acesso legítimo. Usar tentativas limitadas e explicar quando a alternativa não é equivalente. A decisão de ampliar escopo/custo continua com o proprietário.

16.4 CAPTCHA, MFA e takeover

Não burlar CAPTCHA, MFA, paywall, limite de serviço ou bloqueio de acesso. Pausar, identificar o motivo e oferecer intervenção legítima. Ao assumir controle, o proprietário recebe a tela do guest; bloquear cliques do agente, revogar o lease do browser e ocultar captura persistente de campos secretos.

Ao devolver controle: descartar fila de cliques anterior, recapturar página/tela, revalidar sessão e política, comparar o estado com o plano e prosseguir. Exigir um controlador por perfil; pesquisadores paralelos não podem clicar na mesma sessão.

16.5 Proposta de recurso novo

Quando identificar uma ferramenta/assinatura que pode ajudar, preparar CapabilityRequest: problema, capacidade faltante, fornecedor, evidência de que resolve, preço observado, renovação, dados enviados, alternativas gratuitas, risco e plano de teste. Não afirmar “essa assinatura desbloqueia tudo”.

Aprovação de assinatura é diferente de aprovação para enviar dados sensíveis a ela. Se aprovada, criar conta operacional quando permitido, configurar a ferramenta, testar uma tarefa controlada e retomar o objetivo original. Se recusada, continuar com alternativa viável ou entregar limite explicado, sem insistência repetitiva.

17. Instalação de ferramentas e aprendizado de skills

17.1 Descoberta e desenvolvimento de habilidade

Ao precisar de capacidade ausente, consultar catálogo de skills confiáveis; depois documentação oficial e ferramentas permitidas. Pode propor instalar pacote gratuito no guest ou desenvolver uma skill, dentro de mandato técnico e quota. Downloads exigem origem, licença, versão e hash. Não instalar dependência no host por shell genérico.

Código gerado começa na Execution Box sem segredos, sem rede por padrão, com limites de CPU, RAM, disco e tempo. Permissão temporária de rede para baixar dependência é separada da execução. Não disponibilizar chaves de assinatura, tokens principais ou arquivos de produção no teste.

17.2 Manifesto e promoção

Cada skill tem ID, versão, descrição, schema de entrada/saída, capacidades, destinos de rede, formatos, dependências fixadas, limites, testes, evidência, assinatura/hash e rollback. Estados: DRAFT, TESTED, REVIEWED, ACTIVE, QUARANTINED e REVOKED. Ativar só após executar a suíte correspondente.

A promoção sem ampliação de autoridade pode ser automática sob mandato previamente definido, se todos os testes e controles passarem. Ampliação de permissões, custo, rede ou compartilhamento exige aprovação. O LLM não altera Policy Engine, broker, Vault, atualização ou thresholds para passar nos testes.

17.3 Reutilização e rollback

Registrar em quais tarefas a skill foi usada e qual versão produziu cada artefato. Falha repetida pode quarentenar a versão e retornar à anterior compatível, preservando efeitos. Não apagar evidência ruim nem promover habilidade porque o próprio modelo a avaliou positivamente sem teste.

Critério de aceite: uma tarefa encontra capacidade ausente, gera skill de transformação local de dados sintéticos, falha num teste, corrige, passa, é ativada dentro do escopo e depois reutilizada. Uma tentativa de rede ou acesso a credencial fora do mandato deve ser bloqueada. Isso comprova aprendizado de procedimento, não treinamento de pesos.

18. Contas, e-mail e identidade operacional

18.1 Conta do Atlas não é conta pessoal do proprietário

Manter inventário: serviço, propósito, identificador, responsável legal, recuperação, credenciais por referência, termos aceitos, escopos, renovação e estado. Acesso a conta pessoal só mediante integração opcional, específica e revogável. Não usar a sessão já aberta no navegador do proprietário.

Criação de conta gratuita pode ser executada sob mandato válido quando os termos e os dados estiverem cobertos. Se houver telefone, documento, verificação humana, idade, representação ou obrigação material, pedir o que falta. A IA pode preencher dados operacionais reais autorizados; não inventá-los para passar num cadastro.

18.2 E-mail operacional real

Implementar uma conta dedicada ou domínio/caixa autorizado, com API oficial/OAuth preferencialmente. Enviar e receber, anexar entregáveis, registrar recibos, guardar conversas e revogar tokens. O responsável conserva recuperação e administração.

Um e-mail recebido não é uma ordem do dono apenas pelo nome/endereço do remetente. Conteúdo externo é dado não confiável. O proprietário pode ser notificado por e-mail e executar ações em link do companion autenticado. Link de aprovação expira, é ligado à ação e não autoriza sozinho sem sessão.

18.3 Atos externos e dinheiro

Cartão para IA, saldo pré-pago ou conta do fornecedor são integrações opcionais, não substitutos da Policy Engine. Não registrar PAN/CVV ou usar identidade bancária falsa. V1 pode pesquisar e preparar propostas de compra, com confirmação final por driver homologado ou humano. Cada serviço financeiro exige avaliação própria antes de ativar.

Permissão para pesquisar passagem por milhas não concede acesso ao programa nem emissão. Saldo pode ser informado pelo usuário ou consultado por acesso delegado. Assento, disponibilidade e preço precisam de fonte atual e distinção entre cotação e reserva. Nenhum fluxo específico de viagens deve dominar o núcleo generalista.

19. Canal remoto pelo celular

19.1 Companion obrigatório na V1

Criar web companion responsivo com a mesma conversa, tarefas, anexos, perguntas, aprovações permitidas e entregáveis. Não criar uma personalidade nem memória separadas. O Mac mantém autoridade. O relay só recebe e encaminha envelopes autorizados, com filas duráveis e estados.

Pareamento: QR/código de uso único, expiração curta, confirmação local, chave por dispositivo e registro de escopos. Revogação imediata, proteção contra replay e consulta de dispositivos ativos. Não expor SSH, terminal, VNC público ou socket administrativo na internet.

19.2 Segurança e criptografia

Conexão de saída do Mac para relay com autenticação forte. Conteúdo do companion deve ser cifrado ponta a ponta mediante protocolo/biblioteca estabelecidos, incluindo troca/rotação/recuperação de chaves. Não implementar criptografia caseira. TLS por si só não deve ser descrito como E2EE.

Registrar metadados mínimos para entrega e abuso. O relay pode ver os metadados que o desenho exigir; documentar o que vê. Push contém aviso genérico, sem trecho sensível. Uploads remotos têm quotas, validação e origem, e só entram no Core após autenticação e autorização.

19.3 Offline, reenviar e aprovar

Mac offline: companion aceita o pedido se o relay estiver disponível e mostra “aguardando o computador do Atlas”. Não marca em execução até evento real do Core. Retorno online aplica deduplicação, revisões e política atual; assinatura/mandato expirado não é usado só porque era válido no envio.

Controle remoto de parada usa canal prioritário quando o Mac estiver conectado. Se estiver offline, enfileirar a ordem com precedência sobre novos trabalhos e declarar que ainda não foi aplicada. Não prometer interrupção imediata de um computador inacessível.

Operação de alto impacto exige autenticação reforçada em superfície homologada; voz ou mensagem de WhatsApp/e-mail pode notificar, mas não é autorização financeira suficiente por padrão. Testar replay, revogação, dispositivo perdido, aprovação expirada e retomada após falha.

20. Voz: a mesma conversa, sem um segundo agente

20.1 Experiência obrigatória

Botão de falar, indicador de microfone, transcrição visível, resposta falada natural em português brasileiro e possibilidade de continuar por texto. Começar com captura explícita; wake word e escuta permanente ficam desativadas. Voz sintetizada deve ser identificada como IA, sem clonagem de pessoa real no escopo inicial.

Usar `SpeechProvider` para transcrição/síntese e suporte a conversa em tempo real quando homologado. Avaliar o modelo de voz disponível no catálogo e na conta no momento da implementação. Não fixar um ID inventado; integrações de áudio precisam de custo, latência e testes próprios. A documentação de Realtime é referência de transporte, não entrega pronta do orquestrador. [R11]

20.2 Interrupção e intenção

Distinguir “pare de falar” de “pare o trabalho”. Interromper reprodução não necessariamente cancela a tarefa; “pare tudo” deve usar o controle operacional. Durante fala do Atlas, a pessoa pode corrigir ou perguntar sem perder a conversa. Mostrar claramente quando há ruído ou baixa confiança na transcrição.

Valores, datas, nomes de destinatário e aprovações incertas exigem confirmação de conteúdo. Não executar compra por uma transcrição ambígua. Áudio de TV, arquivo anexado ou voz de terceiro é conteúdo externo, não autoridade do dono. Não tratar reconhecimento de voz como autenticação forte por si só.

20.3 Integração, testes e custo

Áudio vira o mesmo envelope durável de mensagem, com referências e classificação. Textos parciais não podem disparar mutações irreversíveis. Confirmar turno e intenção antes da proposta. Gerenciar cancelamento, eco, reconexão e sessões encerradas para não continuar cobrando áudio silencioso.

Testar sotaque brasileiro, nomes portugueses, números, dias da semana, silêncio, interrupção, barulho e troca de dispositivo. Armazenamento de áudio é opt-in; logs não guardam gravações por conveniência de depuração. Medir latência p50/p95 e consumo real autorizado, separando reconhecimento, inferência e síntese.

21. Interface e instalação para pessoa comum

21.1 Onboarding

Abertura inicial deve conduzir: diagnóstico do Mac; nome/idioma; explicação do escritório próprio; modo de inteligência; conta/credencial guiada quando necessária; orçamento; privacidade; preparo do workspace; canal de contato opcional naquele momento; demonstração de baixo risco. Guardar progresso e permitir retomar sem cadastrar tudo outra vez.

Não mostrar terminal, Docker, Python, bancos, YAML, tokens ou nomes de API como requisito cotidiano. Algumas autorizações do sistema e verificação de conta dependem do usuário; explicar por que e abrir o destino correto, sem pedir que desative proteções globais. No modo BYOK, guiar a etapa de conta sem fingir que ela não existe.

Teste com pessoa sem programação: ela deve conseguir instalar, chegar à primeira conversa e receber um arquivo sem ajuda de desenvolvedor. Registrar pontos de confusão; “funciona no meu computador” não é aceite.

21.2 Telas e estados

Conversa: mensagens por papel, anexos, ação de responder, resultados abríveis, voz, contexto de tarefa e botão de parada. Não manter caixa “Como tarefa” como única forma de o produto entender trabalho; ela pode ser avançada/opcional.

Trabalho: lista agrupada por estado, objetivo, prioridade, progresso real, dependências, custo e ações válidas. **Memória:** consultar, confirmar, corrigir, esquecer e exportar. **Arquivos:**

entradas/saídas, busca, versões, prévia e exportação.

Aprovações: ação exata, destinatário, dados, custo, recorrência, prazo, termos, risco e negar sem punição. **Ver trabalhando:** tela real, tarefa ativa, pausa/takeover e indicador de dados sensíveis.

Configurações: identidade, inteligência, orçamento, privacidade, contas, disponibilidade, canais e atualização. **Saúde:** diagnóstico legível e percurso de recuperação.

Estados visuais devem distinguir disponível, executando, aguardando você, pausado, offline, bloqueado e falha. “Conectado” não substitui todos eles. Não usar cor como único indicador; oferecer rótulo, teclado, VoiceOver, foco coerente e contraste.

21.3 Mensagens e sincronização

Upsert por message_id com revisão/sequência; corrigir vínculo de mensagem já carregada. Recuperar todo intervalo desde cursor confirmado, não somente últimas cinquenta mensagens. Atualizar cursor depois de aplicar o lote. Paginação para trás não pode duplicar ou reordenar o histórico.

Preservar rascunho se envio falhar, com envelope completo; não chamar de “não enviado” um pedido já aceito cujo recibo apenas não chegou. Exibir “confirmando recebimento” quando o resultado for desconhecido. Consistência entre UI e Core é requisito funcional.

22. Ciclo de vida, disponibilidade e recuperação do Mac

22.1 Serviços autorizados e independentes

Usar serviço de usuário com mecanismos suportados, avaliando SMLaunchAgent para login helper/LaunchAgent. Registrar permissões e status; o usuário pode desabilitar. A Apple documenta o gerenciamento desses helpers e suas condições de registro. Não requerer daemon root para executar o agente pessoal. [R12]

Fechar janela mantém os serviços quando habilitados. Sair da interface, pausar o funcionário e encerrar tudo são ações diferentes, com texto preciso. No modo com serviço independente, sair da UI não mata o worker; encerrar tudo drena/para e impede religamento até decisão apropriada. Não deixar serviço oculto que reaparece contra a vontade do proprietário.

22.2 Instância única e encerramento

Adquirir trava interprocesso por diretório de dados antes de alterar token, socket ou executar recuperação. Segunda janela se conecta à instância viva. Arquivos órfãos exigem verificação de processo/lock, não remoção indiscriminada de socket.

Encerramento fora do MainActor: pedido cooperativo, checkpoint, prazo, diagnóstico e término forçado controlado se necessário. Preservar UNKNOWN de ações que possam ter acontecido. Não aguardar waitUntilExit sem limite na thread da interface. Recuperar saída parcial na próxima inicialização.

22.3 Suspensão, login e energia

Mac suspenso/desligado não executa o runtime local. Após reinício, sessão e desbloqueio de chaves podem ser necessários. Não habilitar login automático nem retirar FileVault para contornar isso. Agendamento precisa mostrar disponibilidade prevista; relay não é executor.

Opção de evitar suspensão durante tarefa autorizada pode existir com mecanismo suportado, consentimento e indicação de energia; não garantir operação com tampa fechada em todo

hardware. Após retorno, reconciliar estado antes de novos efeitos e informar tarefas que ficaram pendentes.

23. Proteção de armazenamento, backup e atualização

23.1 Dados em repouso

Cifrar banco operacional, arquivos, índices relevantes e backups com bibliotecas auditadas e integração ao Keychain. SQLCipher ou alternativa compatível com as consultas existentes exige prova de build, licença, FTS e restauração. Não inventar cifra nem guardar chave junto ao banco em texto.

Artefatos usam criptografia autenticada, metadados suficientes para versão/nonce e proteção de chaves. Mostrar limites: informação já aberta em memória ou enviada a um provedor não é protegida somente pela cifra do disco. Logs e staging também têm política de retenção e classificação.

23.2 Backup recuperável

Backup consistente inclui banco, artefatos, manifestos e mecanismo autorizado de recuperação das chaves. Testar restauração em instalação limpa, chave ausente, backup corrompido e versão incompatível. Não dizer “backup pronto” sem restaurá-lo em teste.

Restore não restaura autoridade ativa cegamente: revogar/reavaliar sessões, mandatos, aprovações e ações pendentes; reaplicar exclusões/tombstones conforme política. Comparar efeitos externos com realidade atual antes de retomar. Perda definitiva de chave pode impedir recuperação; explicar isso no produto.

23.3 Atualização e supply chain

Separar versões de app, helpers, runtime, guest, browser, skills e schema. Pacotes têm origem, assinatura, hash, compatibilidade e rollback testados. Não substituir automaticamente tudo por latest em cada abertura. Chaves de assinatura de distribuição nunca ficam na máquina do Atlas nem no repositório.

Antes de migração: pausar novos efeitos, persistir operações em andamento e criar recuperação. Rollback de binário não implica rollback seguro de schema; usar matriz de compatibilidade ou restauração conciliada. A atualização deve preservar a identidade e o conhecimento do funcionário.

24. Observabilidade, metas e critérios mensuráveis

24.1 Evidência por operação

Registrar IDs, timestamps, versões, modelo efetivo, duração, custo categorizado, ferramenta, estado, política e referência de evidência. Não registrar senha, token, cookie, prompt completo sensível ou cadeia privada de raciocínio. Redação ocorre antes de gravar logs/exportar diagnóstico.

Timeline do dono usa linguagem comum: “li estes documentos”, “guardo esta aprovação”, “encontrei esta limitação”. Diagnóstico técnico guarda stack/redação, erro normalizado e passos de recuperação. Suporte exportável tem prévia e consentimento.

24.2 Metas propostas, não medições já obtidas

Estabelecer Mac de referência identificado com hardware, macOS, versões e carga. Medir pelo menos trinta amostras de latência onde aplicável. Metas iniciais:

- Reconhecer recebimento persistido local em até 1 s p95, sem incluir a resposta do modelo.
- Parada autenticada impede novos despachos em até 2 s, incluindo envio/chat lento e upload; documentar o ponto de linearização.
- Interface permanece responsiva durante inferência, downloads e shutdown; nenhuma espera de rede ou processo no MainActor.
- Exportação de 1, 10 e 50 MiB tem custo de leitura/hash aproximadamente linear, comprovado por instrumentação.
- Cem mensagens entre memória e consulta não eliminam lembrança; dez arquivos arrastados recebem resultado individual.
- Scheduler é avaliado com tarefa longa simultânea; atraso e tolerância são registrados, sem execução duplicada.
- Idle sem tarefa/agendamento não consome chamadas contínuas ao modelo. Voz encerrada não mantém sessão paga desnecessária.
- Zero violações de autorização, vazamento de sentinela e replay em toda a suíte crítica de release. Isso é critério de teste, não garantia de invulnerabilidade.

Falhar numa meta exige correção ou revisão explícita da meta com medição e impacto; não inventar resultado. Usar dados sintéticos na avaliação pública e guardar evidências sem conteúdo pessoal.

25. Testes que realmente representam o produto

25.1 Camadas obrigatórias

Unitários: política, dinheiro, tempo, normalização, identidade, estados e classificação. Contratos: schemas, framing e compatibilidade Swift/Python/companion. Integração: classes reais, SQLite, Keychain, provider de teste, worker e broker. Recuperação: fault injection entre commits/despachos, restart, redução de limite e concorrência. Segurança: injeção, replay, destino, objetos cruzados, isolamento e supply chain.

E2E: o mesmo bundle e a mesma interface entregues ao usuário. Tests de ViewModel com fake são úteis, mas não substituem upload, pergunta, correção, voz, exportação e restart pelo app. Testes de VM são executados em hardware/capacidade de virtualização realmente disponíveis, não presumidos de um runner genérico.

Avaliações de IA: cenários versionados e respostas avaliadas por critérios; registrar modelo, parâmetros, fontes, custo e número de execuções. A suíte controlada deve cobrir caminhos adversos; modelo real mede comportamento probabilístico. Ambos são necessários.

25.2 Regressões dos 32 achados

Cada A3-01 a A3-32 precisa de triagem, teste no código real, correção ou justificativa e evidência. O anexo A preserva impacto, arquivos e teste obrigatório. Probes antigos são pistas, não testes integrados prontos. Não afirmar que falha condicional foi explorada se apenas foi identificada estaticamente.

Quando viável, capturar execução antes da correção que demonstra o problema e depois que comprova o comportamento certo. Testes não podem ser afrouxados para obter verde.

Reestruturar teste é permitido quando o contrato muda justificadamente; a regressão e a propriedade de segurança devem permanecer cobertas.

25.3 Cenários gerais novos

A suíte executável deve incluir todos os cenários T01–T40 descritos no anexo B. Os cenários críticos são gates, não sugestões. Para IA real, executar ao menos três variações por cenário probabilístico prioritário, registrar falhas e não ocultá-las com média. Meta inicial de 90% de sucesso funcional na amostra é apenas filtro de avaliação: não compensa nenhum erro crítico de segurança, cálculo obrigatório ou execução não autorizada.

Relatórios com julgamento humano indicam revisor e critérios. Capturas de tela precisam mostrar o app real, sem reconstrução gráfica, mock disfarçado ou dado sensível. Falta de Mac/chave bloqueia aquele teste, não autoriza fabricar evidência nem abandonar toda a implementação independente.

26. Plano de construção e gates de conclusão

26.1 Sequência obrigatória

G0 — Baseline e contrato: ler documentos, diagnosticar ambiente, registrar HEAD, preservar alterações, executar suíte disponível e criar matriz dos achados. Saída: baseline factual, backlog ligado a requisitos e primeira regressão implementada. Não reabrir planejamento indefinidamente.

G1 — Controle, privacidade e consistência: classificação/egress, schema completo, comandos prioritários, revisões, heartbeat/watchdog, receipts, transações de entrada, outbox, orçamento vigente, destino de credenciais, escopo de objetos e instância única. Não ampliar poderes antes desses controles.

G2 — Conhecimento, documentos e qualidade: contexto comum, memória temporal/esquecimento, intenções, extratores, leitura paginada, cobertura, critério por evidência e geração de arquivos. Corrigir os falsos positivos de linguagem e o uso de fonte fictícia.

G3 — Alpha local operável: fila de anexos, histórico reconciliado, configurações, botões válidos, início/encerramento e exportação. Executar a jornada completa da aplicação com serviços reais e provedor controlado.

G4 — Inteligência real: mediante autorização de conta/teto, executar cenários do app com modelo real e dados sintéticos; avaliar qualidade, custo, lentidão, cancelamento, correção e retomada. Não substituir essa etapa pelo botão de JSON de saúde.

G5 — Workspace e pesquisa: provar VM, fronteiras, broker guest/host e Execution Box; depois navegador, pesquisa, takeover e evidências. Objetivo composto real deve ser concluído sem passos dados pelo usuário.

G6 — Contas e novas capacidades: conta operacional utilizável, mensagens autorizadas, fluxo de cadastro/solicitação de recurso e skills testadas com promoção/rollback. Pagamentos continuam delimitados por drivers e autorização, não por browser irrestrito.

G7 — Voz e contato remoto: conversa falada, interrupção, transcrição, companion com E2EE, pareamento/revogação, fila offline e entrega real. Conta de e-mail do Atlas pode servir como canal adicional.

G8 — Produto para leigo: onboarding completo, memória/arquivos/contas/configuração, acessibilidade, item de login, disponibilidade, cifra, backup/restauração e diagnóstico. Teste com usuário sem programação.

G9 — Distribuição: build candidata, cadeia de assinatura, notarização, instalador, update/rollback, SBOM/licenças, hashes, changelog e manual. Instalar em Mac limpo compatível, sem toolchain de desenvolvimento. Notarização é etapa formal de distribuição, não teste completo de segurança. [R13]

G10 — Aceite V1: revisão independente dos controles críticos, matriz fechada, evidências de T01-T40, limites conhecidos e aprovação humana. Sem isso, continuar como Alpha/Beta com status verdadeiro. Não renomear Alpha incompleta para V1 apenas porque existe instalador.

26.2 Trabalho paralelo permitido

Podem avançar em paralelo extratores, UX, testes, wrapper de modelos, pesquisa de VM e documentação, desde que branches/contratos não se contradigam. Autoridade, migrações e protocolo exigem coordenação e revisão antes de merge. Nenhum ramo ativa capacidade externa que dependa de gate ainda não aprovado.

Dividir cada pacote do backlog em mudanças revisáveis. Uma única mudança gigante “implementar tudo” é proibida. Preferir testes + implementação + integração por fatia. Não parar para pedir ao proprietário decisões internas de pasta/biblioteca quando há escolha técnica razoável documentável.

26.3 Dependências humanas realmente indispensáveis

Mac compatível e acesso de desenvolvimento; autorização de conta/teto de API; contas operacionais/recuperação; aprovação de termos e fornecedores pagos; infraestrutura e domínio de relay quando necessários; credenciais de assinatura/notarização; confirmação de divulgação sensível; aceite final. Solicitar apenas o item bloqueante, com instrução para leigo.

Sem esses acessos, implementar todos os contratos/adaptadores/testes independentes e registrar exatamente o que ficou não validado. Não pedir a chave em chat ou commit: usar mecanismo seguro de inserção local/secret store autorizado. Não contratar hospedagem ou gastar API só para comprovar um teste sem permissão.

27. Como a IA deve trabalhar e prestar contas

27.1 Ciclo obrigatório por item

Ler implementação e testes afetados; reproduzir; explicar a propriedade a preservar; criar regressão; implementar; executar testes; revisar diferenças; atualizar documentação/rastreabilidade; registrar commit local coerente; avançar ao próximo item desbloqueado. Não prometer execução fora da sessão se o ambiente não oferecer isso.

Se um teste mostrar um defeito novo, criar item no backlog e priorizar impacto. Não esconder falha porque não estava entre os 32 achados. Se uma hipótese da auditoria não se reproduzir, registrar versão, cenário e evidência; não inventar correção desnecessária.

27.2 Relatório exigido, não texto genérico

Para cada item: ID, requisito, classificação anterior, ambiente, arquivos alterados, comando exato, resultado/saída, limites, evidência e commit. Distinguir IMPLEMENTADO, COMPILADO, UNITÁRIO, INTEGRADO, E2E, MODELO_REAL, HARDWARE_REAL e APROVADO. Cada marca exige prova própria.

Nunca escrever “todos os testes passaram” sem contagem, quais suites, ambiente, skips e falhas. Nunca afirmar “sem bugs” ou “à prova de falhas”. Nunca marcar gate por teste que só chama a função falsa que o próprio teste preparou.

27.3 Repositório e publicação

Usar branch de implementação e preservar mudanças do usuário. Não fazer reset destrutivo, force-push, apagar histórico ou reescrever contrato antigo. Merges e releases seguem a autorização vigente do proprietário; esta instrução de engenharia não equivale a autorização para publicar dados ou aceitar custos externos.

CI separa build/tests Python, Swift, integração IPC/Keychain, empacotamento, UI, VM, provider real opt-in e segurança. Job crítico falha se a evidência obrigatória estiver ausente. Screenshots opcionais podem ser informativos, mas não entram como gate verde apenas por continue-on-error.

27.4 Documentos que devem existir ao longo da construção

`BASELINE.md`, `REQUIREMENTS_TRACEABILITY.md`, `AUDIT_REMEDIATION.md`, `CAPABILITY_MATRIX.md`, `TEST_EVIDENCE.md`, `PROGRESS.md`, `EXTERNAL_DEPENDENCIES.md`, `THREAT_MODEL.md`, `MODEL_CATALOG.md`, `BUILD_AND_RUN.md`, `OPERATIONS.md`, `USER_GUIDE_PT_BR.md` e ADRs. Atualizar os existentes em vez de criar cópias divergentes. Os nomes podem ser adaptados com índice claro.

O catálogo de capacidades separa implementado, configurado, autorizado, validado e indisponível. O usuário deve saber o que está funcionando de verdade. Pendências precisam ter causa, próximo passo e se afetam o teste local, a V1 ou apenas uma extensão futura.

28. Evidência de aceite e entrega final

A entrega contém código-fonte, lockfiles, contratos, migrações, testes, catálogo de capacidades, relatório de falhas corrigidas, evidência de isolamento, modelo/uso, manual, backup/restore, instalador realmente gerado e checksum. Arquivos em download precisam existir e corresponder ao commit e à assinatura divulgados.

Verificar no bundle: Python e bibliotecas embutidos; browser/guest provisionáveis; licenças; ausência de dados pessoais, fakes e segredos de desenvolvimento; compatibilidade do macOS declarada; sem dependência do diretório do programador. Assinar executáveis/helpers e componentes conforme exigências efetivas, notarizar, verificar e registrar o resultado. Não orientar desativação global de segurança para fazer a instalação parecer simples.

Aceite humano deve utilizar cenários gerais com dados sintéticos e objetivos diferentes dos usados durante a programação. A pessoa instala, nomeia, conversa, delega, corrige, interrompe, recebe, consulta memória e recupera após falha sem terminal. Comparar o comportamento observado com este contrato, não com a quantidade de linhas de código.

Condição final: o proprietário consegue trabalhar com um funcionário digital persistente, num ambiente próprio, sem organizar manualmente agentes ou repetir contexto já fornecido; o Atlas resolve o que pode, pede somente decisões necessárias e não finge executar o que não consegue comprovar.

29. Configuração de referência e contratos operacionais

29.1 Configuração sem segredos

O exemplo seguinte é um contrato declarativo para implementação, não um arquivo já suportado integralmente pela Alpha. O implementador deve criar schema, migração e UI correspondentes. Todos os limites monetários começam pendentes de decisão do proprietário.

```

schema_version: "2.0"
identity:
  name: Atlas
  locale: pt-BR
  timezone: America/Sao_Paulo
intelligence:
  mode: automatic
  provider: openai
  profiles:
    general: {model_id: gpt-6-sol, requires_account_validation: true}
    deep: {model_id: gpt-6-astra, requires_account_validation: true}
    light: {model_id: gpt-6-luna, requires_account_validation: true}
  cross_provider_fallback: false
  paid_calls_require_setup: true
budget:
  currency: USD
  monthly_limit_minor: null
  per_task_limit_minor: null
  purchases_enabled: false
  alert_percentages: [70, 90]
privacy:
  cloud_sensitive_data: deny_until_scoped_consent
  raw_audio_retention: disabled
  continuous_screen_recording: disabled
  content_logging: redacted
workspace:
  provider: linux_arm64_vm
  host_home_shared: false
  direct_network: false
  generated_code_environment: disposable_vm
runtime:
  max_research_workers: 2
  max_browser_controllers_per_profile: 1
  control_lane: independent
  heartbeat_interval_seconds: 15
  lease_ttl_seconds: 60
remote:
  enabled: false
  require_local_pairing: true
  require_end_to_end_encryption: true
voice:
  enabled_after_consent: true
  always_listening: false

```

Não guardar chaves, cookies ou tokens nesse arquivo. Campos `null` mantêm chamadas pagas bloqueadas. Revalidar IDs/capacidades durante a configuração; preços não estão congelados nesse YAML.

29.2 Protocolo de decisão do Runtime

A decisão tipada deve representar uma entre: responder, propor ferramenta, pedir informação, propor alteração de plano, solicitar capacidade ou terminar com artefatos/evidências. Não carregar campos de aprovação ou risco autodeclarados pelo LLM. O adaptador pode usar JSON estrito com campos nulos exigidos pelo provedor ou chamadas de função nativas, mas o Core deve validar a variante completa.

Antes da decisão: ler tarefa/instruções vigentes e construir contexto autorizado. Depois: validar schema e catálogo, checar tamanho/referências, converter para proposta, conferir política/orçamento e executar pelo broker. Se o modelo produzir texto fora do contrato, rejeitar e reparar com limite. Não executar trechos extraídos por regex de uma resposta arbitrária.

29.3 Prompt-base do Atlas, separado deste contrato

O prompt-base operacional deve instruir: você é o funcionário digital configurado; não é humano; conhece apenas o que foi fornecido ou recuperado; recebe objetivos e propõe meios; não concede permissões; distingue instrução autenticada de conteúdo externo; preserva fontes, vigência e incerteza; consulta memória quando pertinente; pergunta só por informação material ausente; não declara conclusão sem evidência; respeita interrupções; não promete acesso universal; solicita recursos novos de forma fundamentada.

Os fatos pessoais, a identidade configurada, as políticas vigentes e o catálogo não ficam todos fixos nesse texto. Entram por serviços tipados e contexto atual. Autorização e validação continuam fora do prompt mesmo quando o modelo é muito capaz.

30. Referências e controle de fontes

Base do projeto: conversa com o proprietário; Especificação Técnica v1.0; Auditoria da Alpha 2 e prompt de continuidade; consulta de `main` em 24/09/2026. Os achados do anexo A continuam classificados pelo grau de evidência da auditoria, não são apresentados como novos testes executados neste documento.

Os capítulos normativos são decisões de engenharia deste projeto. Referências externas documentam APIs e capacidades gerais; não comprovam implementação nem disponibilidade na conta do proprietário. Esta preparação não executou o Atlas, não fez chamadas faturáveis, não alterou o repositório e não repetiu a auditoria completa.

[R01] OpenAI, catálogo de modelos, original em inglês, consultado em 24/09/2026:
<https://developers.openai.com/api/docs/models>

[R02] OpenAI, GPT-6 Sol, consultado em 24/09/2026:
<https://developers.openai.com/api/docs/models/gpt-6-sol>

[R03] OpenAI, GPT-6 Astra, consultado em 24/09/2026:
<https://developers.openai.com/api/docs/models/gpt-6-astra>

[R04] OpenAI, Structured model outputs, consultado em 24/09/2026:
<https://developers.openai.com/api/docs/guides/structured-outputs>

[R05] OpenAI, Data controls, consultado em 24/09/2026:
<https://developers.openai.com/api/docs/guides/your-data>

[R06] OpenAI, faturamento de ChatGPT e API, consultado em 24/09/2026:
<https://help.openai.com/en/articles/9039756-managing-billing-for-chatgpt-and-the-api-platform>

[R07] Apple, VZVirtioSocketDevice, consultado em 24/09/2026:
<https://developer.apple.com/documentation/virtualization/vzvirtiosocketdevice>

[R08] Apple, VZFileHandleNetworkDeviceAttachment, referência de API a validar no alvo:
<https://developer.apple.com/documentation/virtualization/vzfilehandlenetworkdeviceattachment>

[R09] Playwright, Authentication, consultado em 24/09/2026:
<https://playwright.dev/python/docs/auth>

[R10] Playwright, Browsers, consultado em 24/09/2026:
<https://playwright.dev/python/docs/browsers>

[R11] OpenAI, Realtime API, consultado em 24/09/2026:
<https://developers.openai.com/api/docs/guides/realtime>

[R12] Apple, SMapService e registro, consultados em 24/09/2026:

<https://developer.apple.com/documentation/servicemanagement/smappservice>

[R13] Apple, Notarizing macOS software before distribution, referência de API a validar na release:

<https://developer.apple.com/documentation/security/notarizing-macos-software-before-distribution>

Consulta da baseline:

<https://github.com/ViniciusMilanez82/atlas/tree/d81cece2713ccb5b12e58902e55b05a614467951>

Controle de versão: este contrato v2.0 acrescenta instruções de implementação e testes ao acervo existente. Os anexos seguintes são parte obrigatória do contrato; não são material opcional de leitura.

ANEXO A — 32 ordens de correção obrigatórias

Estas ordens incorporam todos os achados da auditoria da Alpha 2. A natureza da evidência original é preservada: constatação estática, teste isolado ou risco condicional não equivalem a exploração nem teste do aplicativo completo. A IA implementadora deve reproduzir cada falha pertinente nas classes e nos fluxos reais do HEAD antes de classificar seu estado. Nenhum achado começa como corrigido por estar neste documento.

As pastas de teste propostas abaixo são destinos de implementação, não arquivos que afirmamos já existir. Adaptar nomes ao layout vigente sem duplicar suites; manter o identificador A3 na rastreabilidade. Os gates indicam a primeira barreira em que o controle deve estar comprovado; regressões continuam bloqueando os gates posteriores. Em cada item, aplicar também os capítulos citados: eles especificam contratos e invariantes comuns que não devem ser reinventados por correção.

A3-01 — A conversa livre não recupera a memória duradoura

Prioridade: P1. **Gate inicial:** G2. **Detalhamento comum:** capítulos 7–9. **Cenários integrados:** T04.

Pontos de intervenção: `core/conversation.py::_chat`; `runtime/memory/manager.py::search`.

Problema de referência: `_chat` consulta somente as últimas 20 mensagens. Não chama a busca de memórias nem oferece uma ferramenta de recuperação à conversa. A busca existente no `AgentRunner` não supre esse caminho. Uma confirmação de memória pode estar no histórico recente e dar a impressão de lembrança, mas isso não é recuperação duradoura.

O que implementar e como: Criar um serviço de contexto comum a conversa e tarefas: recuperar memórias pertinentes, com classificação, fonte, validade e confirmação. Preservar identidade e preferências aplicáveis mesmo quando o vocabulário da pergunta muda. Não despejar o banco inteiro no modelo.

Teste obrigatório: Guardar e confirmar fato sintético; inserir mais de 20 mensagens; reiniciar; perguntar com paráfrase. Verificar que a fonte correta chega ao contexto, que a resposta a utiliza e que uma memória não confirmada não vira autoridade.

Destino de regressão proposto: `tests/regression/test_a3_01.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-02 — A classificação de dados sensíveis se perde antes da inferência

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 4, 9, 12. **Cenários integrados:** T08.

Pontos de intervenção: `runtime/memory/manager.py::MemoryHit/search`;
`runtime/agent/loop.py::_context/_decide`; `core/conversation.py::_remember/_chat`;
`runtime/models/context.py::ContextItem`; `runtime/models/router.py::candidates`;
`runtime/tools/builtin.py::search_memory`.

Problema de referência: `MemoryHit` e a projeção SQL não carregam sensitivity. O `AgentRunner` transforma o conteúdo recuperado em `ContextItem` com classificação padrão INTERNAL. O requisito de consentimento do roteador olha `data_classification`, normalmente INTERNAL na tarefa. A

conversa também incorpora o texto de confirmações sensíveis no histórico sem preservar essa classificação. Importações iniciam como INTERNAL sem um percurso de classificação no app.

O que implementar e como: Propagar classificação e proveniência em mensagens, memórias, anexos, observações, resumos e entregáveis. Calcular a classificação agregada do contexto efetivamente enviado e aplicar um controle de saída independente do LLM imediatamente antes do transporte. Confirmar uma memória não deve equivaler a autorizar sua divulgação à nuvem.

Teste obrigatório: Usar sentinela sintética SENSITIVE em memória e anexo: sem consentimento, nenhuma chamada ao provedor deve conter a sentinela. Com consentimento escopado, transmitir somente o necessário. Repetir após resumo, correção, retomada, mensagem de confirmação e troca de provedor.

Destino de regressão proposto: tests/regression/test_a3_02.py. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA + SQL ISOLADO. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-03 — Correções em execução não entram no próximo passo

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 5–7. **Cenários integrados:** T19.

Pontos de intervenção: core/conversation.py::handle (correction);
runtime/agent/loop.py::_owner_messages/_resume_state/_context/run.

Problema de referência: A correção é persistida como mensagem vinculada à tarefa e recebe confirmação de que será considerada. Entretanto, as respostas e correções são carregadas em _resume_state; o laço em andamento usa observations já montadas e não consulta novas mensagens a cada passo. A seleção automática da tarefa mais recente ainda pode associar a correção ao trabalho errado.

O que implementar e como: Versionar instruções por tarefa, vincular explicitamente a tarefa alvo e detectar alteração antes de cada inferência e despacho. Uma correção material deve invalidar propostas antigas e exigir replanejamento/reaquisição da autorização pertinente; preservar o que já aconteceu.

Teste obrigatório: Pausar um provedor falso entre decisão e execução, enviar correção por outra conexão e liberar. A ação antiga não pode ocorrer. O próximo contexto deve conter a correção, sem novo task_id e sem repetição de efeitos já confirmados.

Destino de regressão proposto: tests/regression/test_a3_03.py. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-04 — “Pare tudo” pode ser ignorado ou esperar atrás da conversa

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 6, 21–22. **Cenários integrados:** T20.

Pontos de intervenção:

```
platform/macos/AtlasKit/Sources/AtlasKit/AtlasViewModel.swift::stopAll/send/deliver;  
platform/macos/AtlasKit/Sources/AtlasKit/AtlasConnection.swift::call/callSync.
```

Problema de referência: stopAll chama send("pare tudo"). send retorna sem fazer nada quando isSending é true. Além disso, o transporte usa uma única fila serial: comandos de controle compartilham a fila com inferências, leituras e reconexões demoradas.

O que implementar e como: Implementar canal de controle prioritário e independente, que não dependa de isSending, do modelo ou da fila de conversas. Registrar a ordem de parada, revogar leases, interromper ferramentas/chamadas canceláveis e informar efeitos já ocorridos ou incertos.

Teste obrigatório: Com envio pendente, provedor sem resposta, upload em andamento e reconexão, acionar o mesmo botão/atalho do app. Medir recebimento da parada e ausência de novos despachos; não aceitar apenas uma mudança visual ou resposta textual.

Destino de regressão proposto: tests/regression/test_a3_04.py +
platform/macos/AtlasKit/Tests/AtlasKitTests/A304RegressionTests.swift. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-05 — Lease de 60 segundos pode expirar durante uma chamada válida

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 6, 22. **Cenários integrados:** T21.

Pontos de intervenção: runtime/tasks/engine.py::DEFAULT_LEASE_TTL/check_lease_in_txn/heartbeat;
runtime/models/types.py::ModelRequest; runtime/agent/loop.py::run/_run_tool; core/daemon.py::worker.

Problema de referência: O lease padrão é de 60 segundos e o heartbeat ocorre antes da chamada ao modelo. O timeout padrão da requisição também é de 60 segundos, não um prazo total garantido do turno. Depois da expiração, até release/heartbeat são recusados. O worker seleciona apenas CREATED/READY e não reconcilia RUNNING vencido continuamente.

O que implementar e como: Renovar o lease por supervisor independente do trabalho bloqueante, com fencing e limite total; adicionar watchdog de leases expirados que reconcilie efeitos e retome com segurança. Não resolver somente aumentando o TTL.

Teste obrigatório: Com relógio injetável, fazer a inferência ultrapassar o TTL; cobrir resposta tardia, cancelamento, queda e trabalhador obsoleto. A tarefa deve terminar em estado explícito e recuperável, sem ação duplicada nem RUNNING sem heartbeat.

Destino de regressão proposto: tests/regression/test_a3_05.py. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA; REPRODUÇÃO INTEGRADA PENDENTE. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-06 — Resposta estruturada incompleta pode matar o worker sem mudar a saúde

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 4, 6, 11. **Cenários integrados:** T22.

Pontos de intervenção: `runtime/agent/loop.py::_decide/run/_validate`; `core/conversation.py::_chat`; `core/daemon.py::worker`; `core/service.py::_health`.

Problema de referência: O contrato completo DECISION_SCHEMA não é validado localmente antes do uso. `_decide` valida apenas objeto e enum; `summary` nulo ou numérico passa por esse filtro e falha em `[:300]` antes de `_validate`. `_chat` também usa `decision.get` após `json.loads` sem validar o objeto inteiro. O worker captura apenas `AtlasError` e a saúde não verifica a thread.

O que implementar e como: Validar o schema completo, tipos e limites na fronteira antes de qualquer acesso. Acrescentar supervisão da thread/processo, estado de erro recuperável, diagnóstico seguro e reconciliação; não esconder a exceção com sucesso fictício.

Teste obrigatório: Testar JSON como lista, campos ausentes, `summary` `null/int/list`, `question` numérica, decisão inválida e erro interno de ferramenta. Nenhum caso pode matar silenciosamente o worker ou deixar saúde positiva sem executor.

Destino de regressão proposto: `tests/regression/test_a3_06.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA + FRAGMENTO ISOLADO. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-07 — “Entrega verificada” não comprova que o objetivo foi atendido

Prioridade: P1. **Gate inicial:** G2. **Detalhamento comum:** capítulos 13. **Cenários integrados:** T23.

Pontos de intervenção: `runtime/verification/verifier.py::verify_text_artifact/_sources`; `core/daemon.py::worker`; `runtime/agent/loop.py::_complete`.

Problema de referência: O worker usa mínimo de 200 caracteres e uma contagem de fontes baseada no número de anexos. O verificador confere arquivo, hash, leitura, placeholders e presença textual de referências. Não resolve nem relaciona essas fontes ao material real. `_complete` usa essa única evidência para satisfazer todos os critérios requeridos.

O que implementar e como: Separar integridade do arquivo, cobertura do pedido, exatidão de cálculos, existência/proveniência das fontes e revisão de conteúdo. Critérios devem derivar do pedido integral, com evidência por critério. Revisão por outro LLM pode ajudar, mas não é prova factual isoladamente.

Teste obrigatório: Entregar propositalmente assunto errado, cálculos errados, fonte inexistente e cláusula omitida. Cada falha deve impedir conclusão ou produzir entrega parcial claramente rotulada. Um arquivo válido não pode satisfazer automaticamente critérios de negócio distintos.

Destino de regressão proposto: `tests/regression/test_a3_07.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA + VERIFICAÇÕES DE CONTEÚDO ISOLADAS. Se já corrigido no HEAD, apontar o teste e o commit que comprovam

isso; não refazer a correção desnecessariamente.

A3-08 — A palavra portuguesa “todo” é tratada como placeholder

Prioridade: P2. **Gate inicial:** G2. **Detalhamento comum:** capítulos 13. **Cenários integrados:** T23.

Pontos de intervenção: `runtime/verification/verifier.py::PLACEHOLDERS`.

Problema de referência: A expressão usa `(?)` e `\bTODO\b`. Assim, “Todo o trabalho foi concluído” e “Verifique todo o orçamento” são classificados como texto inacabado.

O que implementar e como: Detectar marcadores explícitos contextualizados, como `TODO`: em estrutura de desenvolvimento, sem confundir palavras comuns. A política de placeholders precisa considerar idioma e tipo de artefato.

Teste obrigatório: Aceitar “todo”, “todos” e frases normais; rejeitar `TODO`: completar, [inserir valor] e marcadores claros nos contextos apropriados. Incluir a lista como regressão em português.

Destino de regressão proposto: `tests/regression/test_a3_08.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: REGEX ISOLADA REPRODUZIDA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-09 — Aceitar PDF/Office/imagem não significa extrair seu conteúdo

Prioridade: P1. **Gate inicial:** G2. **Detalhamento comum:** capítulos 10, 14. **Cenários integrados:** T12.

Pontos de intervenção: `platform/macos/AtlasKit/Sources/AtlasApp/AtlasApp.swift::pickFiles`; `runtime/artifacts/manager.py::EXTENSIONS/import_file`; `runtime/tools/builtin.py::read_text`.

Problema de referência: O seletor e o importador aceitam PDF, DOCX, XLSX, PPTX, PNG e JPEG. A ferramenta oferecida ao agente faz decode UTF-8 com `errors="replace"` sobre os bytes brutos. Não há nessa rota extração de páginas, XML Office, células/tabelas ou interpretação visual.

O que implementar e como: Implementar extratores por formato com proveniência (página, célula, seção), limites e falha explícita; usar visão/OCR apenas quando necessário e autorizado. Até então, explicar no app quais formatos são apenas armazenados e quais são compreendidos.

Teste obrigatório: Comparar documentos sintéticos em TXT, PDF com texto, PDF digitalizado e Office; conferir valores em tabelas e cláusulas finais. Arquivo ilegível deve gerar diagnóstico, nunca análise fabricada.

Destino de regressão proposto: `tests/regression/test_a3_09.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTATICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-10 — Documentos e observações longos são truncados silenciosamente

Prioridade: P1. **Gate inicial:** G2. **Detalhamento comum:** capítulos 10, 14. **Cenários integrados:** T13.

Pontos de intervenção: `runtime/tools/builtin.py::read_text;`
`runtime/agent/loop.py::_run_tool/_observe.`

Problema de referência: A leitura limita text a 200.000 caracteres e o AgentRunner corta `json.dumps(output)` em 20.000. O segundo corte pode interromper até o JSON e remove o restante do documento. A ferramenta não oferece offset/paginação para recuperar os trechos perdidos.

O que implementar e como: Guardar o conteúdo completo como artefato e retornar referências com paginação/ranges, tamanho total e `has_more`. Persistir observações estruturadas válidas; o contexto pode resumir, mas a fonte precisa continuar acessível. Exigir cobertura dos trechos relevantes antes de concluir.

Teste obrigatório: Colocar a informação decisiva depois dos caracteres 20.000 e 200.000. O agente deve recuperá-la por outra chamada, registrar o intervalo e não concluir que leu o documento completo só porque leu o prefixo.

Destino de regressão proposto: `tests/regression/test_a3_10.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA + CORTE DE TEXTO ISOLADO. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-11 — Deduplicação de mensagem não recupera processamento interrompido

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 5. **Cenários integrados:** T16.

Pontos de intervenção: `core/conversation.py::handle;`
`platform/macos/AtlasKit/Sources/AtlasKit/AtlasAPI.swift::send;`
`platform/macos/AtlasKit/Sources/AtlasKit/AtlasConnection.swift::callSync.`

Problema de referência: A mensagem do proprietário é gravada antes de a intenção ser processada. Se houver queda nesse intervalo, o reenvio encontra a mensagem e devolve `duplicate`, possivelmente com `reply=null`, sem concluir o processamento. A consulta de duplicidade e a inserção também não formam uma única operação serializada.

O que implementar e como: Criar registro durável de requisição com hash do payload e estados RECEIVED/PROCESSING/COMPLETED/UNKNOWN. O reenvio deve devolver o resultado definitivo ou retomar a requisição incompleta com reconciliação. Mesmo ID com payload diferente deve ser conflito explícito.

Teste obrigatório: Injetar queda depois da mensagem, depois da inferência, antes/depois da criação da tarefa e antes da resposta. O mesmo `client_message_id` deve levar a um único resultado recuperável, não a um reconhecimento vazio permanente.

Destino de regressão proposto: `tests/regression/test_a3_11.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM,

teste de Python isolado não substitui validação macOS. Estado original: ESTATICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-12 — Tarefa pode ser executada antes de todos os anexos serem vinculados

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 4–5. **Cenários integrados:** T17.

Pontos de intervenção: `core/conversation.py::_delegate`; `core/service.py::_task_create`; `runtime/tasks/engine.py::create`; `core/daemon.py::worker`.

Problema de referência: `TaskEngine.create` confirma a tarefa `CREATED` em uma transação. O vínculo dos anexos e da mensagem é feito depois, em outra transação. O worker já pode selecionar `CREATED` entre os dois commits.

O que implementar e como: Preparar objetivo, mensagem, anexos e critérios atomicamente, publicando a tarefa executável somente no último commit; ou usar um estado de preparação não elegível ao worker. Validar a operação inteira antes de expor o trabalho.

Teste obrigatório: Usar duas conexões e barreiras de sincronização para forçar a seleção entre commits. O worker não pode adquirir a tarefa antes de seu pacote de entrada estar completo.

Destino de regressão proposto: `tests/regression/test_a3_12.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTATICA; CORRIDA A REPRODUZIR. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-13 — Roteamento por palavras-chave captura perguntas que não são status

Prioridade: P2. **Gate inicial:** G2. **Detalhamento comum:** capítulos 7. **Cenários integrados:** T03, T10.

Pontos de intervenção: `core/conversation.py::_STATUS/_MEMORY/handle`.

Problema de referência: A regex de status aceita “Como está o tempo no Rio?”, “Qual o andamento da economia?” e “Como vai funcionar esse software?” como consulta de tarefas. A de memória também aceita prefixos parciais: “Lembrete:” vira conteúdo “te: ...”, e formas acentuadas podem manter o prefixo no texto guardado.

O que implementar e como: Manter comandos exatos para parada e ações de autoridade; para intenções gerais usar contexto, limites lexicais e desambiguação. Preservar o texto original e normalizar apenas para reconhecimento, sem perder a extensão correta do trecho capturado.

Teste obrigatório: Criar testes de paráfrases em português: estado de tarefa versus tempo/economia/explicação do software; memória versus lembrete. Perguntas semelhantes com intenções diferentes devem seguir rotas diferentes.

Destino de regressão proposto: `tests/regression/test_a3_13.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: REGEX ISOLADA

REPRODUZIDA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-14 — Pergunta pendente captura assuntos novos como se fossem respostas

Prioridade: P2. **Gate inicial:** G2. **Detalhamento comum:** capítulos 7. **Cenários integrados:** T10.

Pontos de intervenção: `core/conversation.py::_pending_question/handle/_answer;`
`platform/macos/AtlasKit/Sources/AtlasKit/AtlasViewModel.swift::send.`

Problema de referência: Quando há exatamente uma pergunta pendente, mensagens seguintes são tratadas como resposta antes da rota de status/memória/correção. Com mais de uma pergunta, não há seleção automática confiável. A interface não oferece uma associação explícita por pergunta no envio comum.

O que implementar e como: Oferecer responder à pergunta/tarefa por ID na interface e no contrato. Inferir continuidade somente quando inequívoca; assunto novo, status e comandos explícitos não devem ser consumidos como resposta.

Teste obrigatório: Testar uma e duas perguntas simultâneas, resposta fora de ordem, pedido novo, pergunta de status e resposta a pergunta já resolvida. Só a tarefa correta pode ser retomada.

Destino de regressão proposto: `tests/regression/test_a3_14.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-15 — Anexar um arquivo força delegação mesmo quando o usuário só compartilha

Prioridade: P2. **Gate inicial:** G2. **Detalhamento comum:** capítulos 7, 14. **Cenários integrados:** T11.

Pontos de intervenção: `platform/macos/AtlasKit/Sources/AtlasKit/AtlasViewModel.swift::deliver;`
`core/conversation.py::_handle/_delegate.`

Problema de referência: A UI envia `delegate || !ids.isEmpty`. Portanto, a presença de anexo decide que haverá tarefa, independentemente do significado da mensagem. O modo explícito pode anteceder o tratamento de memória ou de uma resposta a pergunta.

O que implementar e como: Separar importação/armazenamento, referência em conversa, gravação de memória e delegação. Não converter anexo em autorização tácita para criar tarefa. Vincular anexos a respostas/correções na tarefa alvo.

Teste obrigatório: Enviar arquivo para guardar, arquivo como complemento de tarefa e arquivo para análise nova. Os três cenários precisam produzir estados e vínculos diferentes, sem tarefas extras.

Destino de regressão proposto: `tests/regression/test_a3_15.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA. Se já corrigido no

HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-16 — Corrigir uma memória remove sua validade temporal

Prioridade: P2. **Gate inicial:** G2. **Detalhamento comum:** capítulos 8. **Cenários integrados:** T06.

Pontos de intervenção: `runtime/memory/manager.py::correct/search`.

Problema de referência: `correct` cria uma nova versão sem `valid_from` e `valid_until`. A busca passa a usar a versão atual, cujos limites ficam nulos, em vez da janela anterior.

O que implementar e como: Preservar a janela anterior por padrão e permitir uma alteração temporal explícita, versionada e validada. Correção de texto não deve mudar silenciosamente a vigência.

Teste obrigatório: Corrigir memória expirada e memória futura. As duas devem continuar respeitando a janela original. Incluir um teste separado para alteração de vigência autorizada.

Destino de regressão proposto: `tests/regression/test_a3_16.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: SQL ISOLADO REPRODUZIDO. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-17 — Excluir memória não elimina cópias usadas no contexto

Prioridade: P2. **Gate inicial:** G2. **Detalhamento comum:** capítulos 8–9, 23. **Cenários integrados:** T07.

Pontos de intervenção: `runtime/memory/manager.py::delete`; `core/conversation.py::_remember/_chat`; `runtime/agent/loop.py::_observe/_resume_state`.

Problema de referência: `delete` limpa `memory_versions` e o índice, mas o conteúdo pode continuar nas mensagens de pedido/confirmação e em observações persistidas. A conversa lê essas mensagens novamente. A interface também não oferece o percurso completo de gerenciar e excluir memória.

O que implementar e como: Definir escopos explícitos de exclusão e tombstones por origem; impedir reintrodução por histórico/resumo/observação. Oferecer revisão e exclusão na interface, explicando limites de backups. Não prometer apagar cópias externas fora do controle do produto.

Teste obrigatório: Guardar, confirmar, usar, excluir e perguntar novamente em seguida e após reiniciar. Inspecionar todo contexto enviado. Testar exclusão parcial versus exclusão completa autorizada e política para backups.

Destino de regressão proposto: `tests/regression/test_a3_17.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTATICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-18 — O contexto pode preservar instruções antigas e descartar a atual

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 8-9, 11. **Cenários integrados:** T09.

Pontos de intervenção: `runtime/models/context.py::ContextBuilder.build;`
`core/conversation.py::_chat;` `runtime/agent/loop.py::_context.`

Problema de referência: A seleção de contexto é gulosa por autoridade e ordem de inserção. Na conversa, instruções anteriores têm a mesma autoridade da última e vêm primeiro. Em histórico longo, a mensagem atual pode cair em `dropped`. Respostas anteriores do modelo ainda são marcadas `VERIFIED_FACT`, sem verificação própria.

O que implementar e como: Reservar espaço para a mensagem vigente e restrições ativas; selecionar por relevância e recência dentro de cada autoridade. Resumir histórico sem transformar saídas do modelo em evidência verificada. Bloquear a inferência ou avisar se o contexto obrigatório não couber.

Teste obrigatório: Saturar o histórico com mensagens longas e inserir uma correção curta no fim. A última instrução precisa estar presente em todas as chamadas. Sentinelas de texto antigo não confirmado não podem surgir como fato verificado.

Destino de regressão proposto: `tests/regression/test_a3_18.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar `linguagens/processos`.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: FRAGMENTO ISOLADO + ESTÁTICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-19 — Reduzir orçamento não atualiza clientes já em execução

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 12. **Cenários integrados:** T24.

Pontos de intervenção: `core/intelligence.py::_client/build_client;`
`core/daemon.py::_components/worker;`
`security/budget/budget.py::BudgetLimits/BudgetManager.reserve_in_txn;`
`runtime/models/router.py::call.`

Problema de referência: `BudgetLimits` é um snapshot mantido no `BudgetManager`. O cliente do modelo é criado antes de rodar a tarefa; o broker também guarda seus próprios limites desde `_components`. Salvar uma revisão de settings não atualiza esses objetos nem os obriga a revalidar a política antes da próxima reserva.

O que implementar e como: Ler a revisão de política/orçamento vigente dentro da transação de cada reserva e despacho. Definir como reduções afetam reservas existentes e cancelar novos gastos incompatíveis; não apagar consumo já realizado.

Teste obrigatório: Executar tarefa em múltiplos passos, reduzir teto por outra conexão entre dois passos e verificar que a chamada seguinte é barrada pelo novo limite. Testar aumento, revogação de consentimento e concorrência com Testar inteligência.

Destino de regressão proposto: `tests/regression/test_a3_19.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar `linguagens/processos`.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA; TESTE

CONCORRENTE PENDENTE. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-20 — Transporte HTTP precisa vincular credenciais ao destino final

Prioridade: P1. **Gate inicial:** G1. **Detalhamento comum:** capítulos 12, 15–16. **Cenários integrados:** T26.

Pontos de intervenção: `runtime/models/openai_responses.py::__init__/_request`.

Problema de referência: O cliente usa `urlopen` com `Authorization` e sem política própria de redirecionamento. A construção isolada de `redirect` da biblioteca Python local manteve o header ao mudar de origem. A exceção HTTP para testes usa `startswith("http://127.0.0.1")`, aceitando também `hostname` com esse prefixo que não é `loopback`.

O que implementar e como: Analisar `scheme/host/porta` com parser; permitir `loopback` exato apenas em modo de testes. Recusar `redirects` ou revalidar origem sem repassar `Authorization` entre origens; vincular uso da credencial ao destino efetivo e rejeitar endpoint arbitrário na distribuição normal.

Teste obrigatório: Usar servidores locais de teste para 301/302/303/307/308: o destino de outra origem não pode receber `Authorization`. Testar prefixos enganosos, downgrade HTTPS/HTTP, URL com usuário e porta. Nunca usar segredo real nesses testes.

Destino de regressão proposto: `tests/regression/test_a3_20.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA + CONSTRUÇÃO DE REQUEST ISOLADA; CONDICIONAL. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-21 — Arrastar vários arquivos pode importar apenas o primeiro

Prioridade: P2. **Gate inicial:** G3. **Detalhamento comum:** capítulos 14, 21. **Cenários integrados:** T14.

Pontos de intervenção:

```
platform/macos/AtlasKit/Sources/AtlasApp/AtlasApp.swift::ConversationView.onDrop;  
platform/macos/AtlasKit/Sources/AtlasKit/AtlasViewModel.swift::attach.
```

Problema de referência: `onDrop` inicia uma `Task` independente para cada arquivo. `attach` retorna silenciosamente se `isAttaching` já é `true`. Como a importação suspende em `awaits`, chamadas seguintes podem chegar enquanto a primeira ainda está ativa. O seletor de arquivos usa um laço sequencial e não sofre a mesma corrida.

O que implementar e como: Usar fila de importação com resultado por arquivo, cancelamento e resumo de erros. Nunca descartar arquivos silenciosamente por estar ocupado.

Teste obrigatório: Arrastar dez arquivos com transporte artificialmente lento: todos devem aparecer como importados ou falhar explicitamente. Repetir pelo seletor e misturar arquivos inválidos.

Destino de regressão proposto: `tests/regression/test_a3_21.py` + `platform/macos/AtlasKit/Tests/AtlasKitTests/A321RegressionTests.swift`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM,

teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA; REPRODUÇÃO VISUAL PENDENTE. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-22 — Reenvio não conserva todo o payload original

Prioridade: P2. **Gate inicial:** G1. **Detalhamento comum:** capítulos 5, 14, 21. **Cenários integrados:** T15.

Pontos de intervenção:

```
platform/macos/AtlasKit/Sources/AtlasKit/AtlasViewModel.swift::failedDraft/retryFailedSend/deliver;  
platform/macos/AtlasKit/Sources/AtlasKit/AtlasAPI.swift::importFile.
```

Problema de referência: failedDraft conserva texto, clientId e delegate, mas não uma lista imutável de anexos/replyTo. O reenvio recalcula os IDs a partir dos anexos atualmente na tela. Uma importação com resposta perdida também não possui uma consulta de recibo que recupere o artifact_id já criado.

O que implementar e como: Persistir um envelope imutável de envio com texto, anexos, tarefa/resposta alvo, intenção, hash e idempotency key. Implementar recibo consultável da importação. Alterar anexos deve gerar novo envelope, não reutilizar o ID anterior.

Teste obrigatório: Derrubar conexão após gravar mensagem/importar arquivo, alterar anexos na UI e tentar novamente. Recuperar o pedido original ou apresentar conflito claro; não criar cópias órfãs ou reinterpretar o payload.

Destino de regressão proposto: tests/regression/test_a3_22.py +
platform/macos/AtlasKit/Tests/AtlasKitTests/A322RegressionTests.swift. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-23 — Atualização do histórico não reconcilia mensagens existentes nem lacunas

Prioridade: P2. **Gate inicial:** G3. **Detalhamento comum:** capítulos 5, 21. **Cenários integrados:** T03, T15.

Pontos de intervenção:

```
platform/macos/AtlasKit/Sources/AtlasKit/AtlasViewModel.swift::merge/refresh/loadOlder;  
core/conversation.py::_set_kind/history.
```

Problema de referência: merge apenas acrescenta IDs novos. Alterações posteriores de kind/task_id em uma mensagem já carregada não chegam à tela. refresh lê somente a página mais recente (50 por padrão), enquanto o cursor de eventos é avançado; um intervalo maior de mensagens pode ficar ausente no meio do histórico local.

O que implementar e como: Fazer upsert por ID com versão/sequence e ordenar por sequência estável. Preencher explicitamente intervalos perdidos desde o último cursor, em vez de apenas anexar a última página.

Teste obrigatório: Mudar kind/task_id de mensagem visível, reconectar depois de mais de 50 mensagens e paginar para trás. Verificar ausência de duplicação, lacunas e ordem incorreta.

Destino de regressão proposto: `tests/regression/test_a3_23.py` + `platform/macos/AtlasKit/Tests/AtlasKitTests/A323RegressionTests.swift`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-24 — Uploads têm coordenação e limpeza incompletas

Prioridade: P2. **Gate inicial:** G1. **Detalhamento comum:** capítulos 5, 14. **Cenários integrados:** T14, T15.

Pontos de intervenção: `core/service.py::__init__/_upload/_import/_upload_path;`
`core/daemon.py::service.`

Problema de referência: Cada `CoreService` tem seu próprio `_uploads_lock`, embora diferentes conexões possam operar no mesmo arquivo de staging. A checagem de tamanho/offset e a escrita não estão coordenadas entre serviços. O staging tem limite por arquivo, mas não expiração/limite agregado demonstrados nesse percurso.

O que implementar e como: Coordenar uploads por ID entre conexões, com metadados transacionais, estado final e recibo. Definir quota agregada, prazo de expiração e limpeza recuperável; usar escrita segura e não tratar staging como arquivo já aprovado.

Teste obrigatório: Enviar o mesmo chunk simultaneamente por duas conexões, fechar a conexão antes de importar, reiniciar e reenviar. Conferir hash final, uma única importação e remoção dos temporários expirados.

Destino de regressão proposto: `tests/regression/test_a3_24.py`. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA; TESTE CONCORRENTE PENDENTE. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-25 — Leitura em chunks relê e recalcula hash do arquivo inteiro

Prioridade: P3. **Gate inicial:** G3. **Detalhamento comum:** capítulos 14. **Cenários integrados:** T32.

Pontos de intervenção: `core/service.py::_read;` `runtime/artifacts/manager.py::read_bytes;`
`platform/macos/AtlasKit/Sources/AtlasKit/AtlasAPI.swift::readArtifact.`

Problema de referência: Para cada chunk de 512 KiB, `_read` chama `read_bytes`, que lê e verifica novamente o arquivo completo. Um arquivo de 50 MiB exige cerca de 100 chunks: aproximadamente 5.000 MiB de leitura/hash cumulativos, além da transferência. Não significa 5 GiB simultaneamente na RAM.

O que implementar e como: Abrir uma sessão de leitura de artefato imutável com verificação inicial, tamanho/hash fixos e leitura por intervalo. Preservar a verificação final e detectar adulteração sem repetir trabalho integral a cada chunk.

Teste obrigatório: Instrumentar bytes lidos/hasheados para 1, 10 e 50 MiB. O custo deve crescer aproximadamente com o tamanho do arquivo, não com tamanho multiplicado pelo número de

chunks.

Destino de regressão proposto: tests/regression/test_a3_25.py. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA + CÁLCULO. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-26 — Algumas referências por ID não validam o proprietário completo

Prioridade: P2. **Gate inicial:** G1. **Detalhamento comum:** capítulos 3-4, 9, 12. **Cenários integrados:** T27.

Pontos de intervenção: core/conversation.py::handle (delegate existing message); core/service.py::_mem_correct; runtime/memory/manager.py::correct.

Problema de referência: A delegação de mensagem anterior usa get_message e confere role, mas não prova que a mensagem pertence à conversa/empregado da sessão. Em correct, o teste de proprietário de memória é mais fraco que em confirm/delete; o caminho de serviço também não liga explicitamente memory_id à sessão.

O que implementar e como: Aplicar verificação central por objeto: session -> owner -> employee -> conversation/task/memory/source. Conferir IDs relacionados em todas as operações, não só nos métodos mais visíveis.

Teste obrigatório: Criar dois proprietários e dois empregados com sessões distintas e tentar delegar mensagem/corrigir memória cruzadas. Tudo deve falhar sem revelar conteúdo, inclusive quando a fonte também é fornecida por ID.

Destino de regressão proposto: tests/regression/test_a3_26.py. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA; AMEAÇA CONDICIONAL A MÚLTIPLAS IDENTIDADES. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-27 — Notificação pode se perder depois de a tarefa mudar de estado

Prioridade: P2. **Gate inicial:** G1. **Detalhamento comum:** capítulos 5, 19. **Cenários integrados:** T18.

Pontos de intervenção: runtime/agent/loop.py::_tell_owner/run/_complete; core/conversation.py::notify.

Problema de referência: A tarefa muda para WAITING_USER ou COMPLETED antes de sua mensagem ser entregue. _tell_owner captura toda exceção e ignora. Não há outbox durável ligada à transação de estado nesse caminho.

O que implementar e como: Persistir evento de notificação na mesma transação da mudança de estado, com chave idempotente e dispatcher recuperável. Uma falha de notificação não deve desfazer um efeito externo, mas precisa ficar pendente e visível para nova entrega.

Teste obrigatório: Falhar antes/depois do commit e durante notify; reiniciar. A pergunta/entrega deve aparecer exatamente uma vez, vinculada à tarefa, e nunca desaparecer silenciosamente.

Destino de regressão proposto: tests/regression/test_a3_27.py. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-28 — Inicialização não demonstra exclusão mútua entre instâncias

Prioridade: P2. **Gate inicial:** G1. **Detalhamento comum:** capítulos 22. **Cenários integrados:** T28.

Pontos de intervenção:

```
platform/macos/AtlasKit/Sources/AtlasKit/Supervisor.swift::start/launchCore;  
core/daemon.py::bootstrap/main.
```

Problema de referência: O Supervisor protege seu próprio objeto com NSLock, mas recria token e remove sockets em caminhos compartilhados ao iniciar. Não há nesses percursos uma trava interprocesso de instância/diretório. Iniciar duas cópias ou um daemon duplicado precisa ser tratado explicitamente.

O que implementar e como: Adquirir trava interprocesso exclusiva sobre o diretório de dados antes de alterar tokens, sockets ou recuperar tarefas; distinguir instância viva de arquivos órfãos. A segunda janela deve conectar-se à instância existente ou recusar com diagnóstico.

Teste obrigatório: Abrir duas cópias e iniciar dois Supervisores/daemons simultaneamente. Deve haver um único responsável pela recuperação/execução e nenhuma remoção do socket de processo saudável.

Destino de regressão proposto: tests/regression/test_a3_28.py. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA; TESTE DE DUAS INSTÂNCIAS PENDENTE. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-29 — Validação da inteligência sobrevive indevidamente à troca de credencial

Prioridade: P2. **Gate inicial:** G1. **Detalhamento comum:** capítulos 11-12. **Cenários integrados:** T29.

Pontos de intervenção: core/intelligence.py::status/register_key/_run_check.

Problema de referência: status encontra uma validação aprovada por provider/model_id. Não a associa à referência/versão da credencial atual nem à base URL utilizada. Trocar a chave pode manter o indicador Pronta baseado em um teste de outra conta.

O que implementar e como: Vincular validação a credencial, endpoint, modelo, capacidades e versão relevante de configuração. Invalidar ou exigir nova validação ao mudar esse vínculo, sem confundir acesso ao modelo com verificação de preço.

Teste obrigatório: Validar uma credencial sintética A em servidor controlado, substituí-la por B sem acesso e conferir que Pronta é retirada imediatamente; repetir mudando endpoint/modelo.

Destino de regressão proposto: tests/regression/test_a3_29.py. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTATICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-30 — Os botões de controle não refletem os estados realmente aceitos

Prioridade: P2. **Gate inicial:** G3. **Detalhamento comum:** capítulos 6, 21. **Cenários integrados:** T30.

Pontos de intervenção: platform/macos/AtlasKit/Sources/AtlasApp/AtlasApp.swift::TaskRow; runtime/tasks/engine.py::resume/_resume_target; core/daemon.py::worker.

Problema de referência: A UI oferece Pausar/Retomar/Cancelar em todos os estados. resume aceita somente PAUSED. Uma tarefa BLOCKED, por exemplo após orçamento esgotado, não é retomada pelo botão sem percurso adicional. Alguns estados de origem podem ser restaurados sem pertencer ao conjunto que o worker seleciona.

O que implementar e como: Derivar ações disponíveis da máquina de estados; criar percurso explícito de desbloqueio que revalide causa, orçamento e efeitos incertos. Não liberar compras ou redespachar UNKNOWN só para fazer o botão funcionar.

Teste obrigatório: Testar os controles em cada estado, incluindo bloqueio de orçamento, aprovação pendente, lease vencido e resultado incerto. Só mostrar ações válidas e documentar o resultado de cada uma.

Destino de regressão proposto: tests/regression/test_a3_30.py + platform/macos/AtlasKit/Tests/AtlasKitTests/A330RegressionTests.swift. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTATICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-31 — Encerrar serviços ainda pode bloquear a interface sem prazo

Prioridade: P2. **Gate inicial:** G3. **Detalhamento comum:** capítulos 22. **Cenários integrados:** T31.

Pontos de intervenção:

platform/macos/AtlasKit/Sources/AtlasApp/AtlasApp.swift::AppController.shutdown;
platform/macos/AtlasKit/Sources/AtlasKit/Supervisor.swift::stop.

Problema de referência: A melhoria assíncrona do IPC não cobre shutdown: AppController no MainActor chama Supervisor.stop, que usa waitUntilExit sem prazo para os processos. Um serviço que não encerre pode prender o menu de encerramento ou a saída do app.

O que implementar e como: Executar shutdown fora do MainActor, com etapas limitadas: parada cooperativa, checkpoint/reconciliação, diagnóstico e término forçado controlado quando indispensável. Não ocultar que o resultado de uma ação externa pode ser desconhecido.

Teste obrigatório: Substituir serviço por processo de teste que ignora a primeira solicitação de término. O app deve continuar responsivo e concluir ou explicar a saída dentro de um limite

mensurável.

Destino de regressão proposto: tests/regression/test_a3_31.py + platform/macos/AtlasKit/Tests/AtlasKitTests/A331RegressionTests.swift. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA; REPRODUÇÃO MAC PENDENTE. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

A3-32 — Salvar resultado não preserva o nome e a extensão do arquivo

Prioridade: P3. **Gate inicial:** G3. **Detalhamento comum:** capítulos 14, 21. **Cenários integrados:** T32.

Pontos de intervenção: platform/macos/AtlasKit/Sources/AtlasApp/AtlasApp.swift::Bubble.saveAs; platform/macos/AtlasKit/Sources/AtlasKit/AtlasViewModel.swift::export.

Problema de referência: O painel Salvar como inicia sempre com nameFieldStringValue="entrega", sem carregar nome/extensão do artefato. A exportação grava no caminho escolhido, mesmo quando a extensão original era .md, .csv ou .json.

O que implementar e como: Carregar metadados do artefato para sugerir o nome completo, extensão e tipo; preservar liberdade de renomear sem trocar silenciosamente o formato.

Teste obrigatório: Exportar TXT, Markdown, CSV e JSON sem editar o nome sugerido. Conferir extensão, conteúdo, hash e abertura no aplicativo indicado.

Destino de regressão proposto: tests/regression/test_a3_32.py + platform/macos/AtlasKit/Tests/AtlasKitTests/A332RegressionTests.swift. Escrever testes de contrato e integração adicionais quando a correção atravessar linguagens/processos.

Evidência para encerrar: registrar commit, reprodução anterior, alteração, comando e saída do teste de regressão, teste integrado relacionado e limitação remanescente. Para UI/Keychain/VM, teste de Python isolado não substitui validação macOS. Estado original: ESTÁTICA. Se já corrigido no HEAD, apontar o teste e o commit que comprovam isso; não refazer a correção desnecessariamente.

ANEXO B — 40 cenários de aceite do produto

Executar estes cenários por camadas: testes determinísticos nas classes reais, integração dos componentes e jornada no bundle. Modelos falsos são apropriados para controle de falhas, mas não homologam qualidade de IA real. Quando um cenário requer Mac, fornecedor ou dispositivo, ausência desse recurso é BLOQUEADO, não PASSOU. Definir gabaritos antes de executar; dados devem ser sintéticos por padrão. Resultados de código, modelo e interface devem estar ligados ao mesmo commit e aos mesmos contratos.

Os limiares do capítulo 24 são metas iniciais. Registrar ambiente, amostra, resultados e falhas; alterar uma meta exige justificativa, não maquiagem do relatório. Qualquer quebra de autorização, perda silenciosa de pedido, ação externa duplicada ou saída proibida bloqueia aceite, ainda que outros testes passem.

T01 — Instalação em Mac limpo

Preparar: Mac Apple Silicon incluído na matriz de suporte, sem Python, Docker, editor ou SDK de desenvolvimento instalado.

Executar: Instalar o bundle candidato pelo percurso gráfico, abrir e executar o primeiro diagnóstico.

Resultado exigido: O aplicativo inicia seus componentes empacotados, informa requisitos reais e permite continuar sem terminal. Verificar origem, assinatura, notarização quando a entrega for de distribuição e erro acionável para hardware não suportado.

Evidência mínima: Bundle real, Mac limpo; vídeo ou passos com capturas reais, manifesto, hash e diagnóstico.

T02 — Identidade e separação da vida pessoal

Preparar: Pastas, cookies, contas e mensagens-sentinela fictícias na sessão pessoal do Mac.

Executar: Criar o funcionário, escolher nome/idioma e importar explicitamente apenas um arquivo.

Resultado exigido: A identidade persiste; somente a cópia escolhida fica disponível. Nenhuma conta pessoal, pasta inteira ou sessão é herdada. Guest e código experimental não obtêm as sentinelas.

Evidência mínima: Teste de autorização e de isolamento, app real e inventário de acessos negados.

T03 — Conversa natural, intenção e histórico completo

Preparar: Conversa com saudações, perguntas explicativas, consultas de andamento e mais de 100 mensagens.

Executar: Alternar assuntos, delegar, reconectar após mais de 50 mensagens e modificar o vínculo de uma mensagem existente.

Resultado exigido: Saudação não cria tarefa; consulta sobre tempo/economia não vira status interno. Histórico recebe atualizações por versão, sem buracos ou duplicações. Pergunta sobre trabalho usa estado real, não invenção do modelo.

Evidência mínima: Corpora pt-BR versionados, gabarito de intenção, eventos e interface.

T04 — Memória fora da janela recente

Preparar: Fato e preferência sintéticos confirmados; mais de 100 mensagens posteriores.

Executar: Reiniciar o aplicativo e perguntar com palavras diferentes, sem repetir o fato no pedido.

Resultado exigido: A recuperação traz a fonte correta; a resposta usa a informação vigente. O teste inspeciona o contexto enviado, não apenas uma coincidência na resposta. Memória proposta não recebe autoridade de confirmação.

Evidência mínima: Recuperação determinística e avaliação com modelo real quando autorizada.

T05 — Correção de preferência e contradição

Preparar: Preferência antiga, nova correção confirmada e uma fonte externa conflitante.

Executar: Solicitar entrega posterior e consultar a razão da escolha.

Resultado exigido: Nova versão prevalece; fonte externa não altera preferência do dono; histórico de substituição existe. Conflito factual material não resolvido gera pergunta específica, sem eliminar evidências.

Evidência mínima: Banco, índice, contexto e resultado com referências de versão.

T06 — Validade temporal, fusos e exceções

Preparar: Memórias futuras/expiradas, eventos de dia inteiro, compromissos em fusos distintos e uma recorrência com exceção.

Executar: Corrigir só o texto de uma memória e cruzar intervalos antes/depois das datas-limite.

Resultado exigido: Validade não desaparece na correção. Sobreposição e exceções são calculadas por código; instantes UTC e datas locais não são confundidos. Calendário é um exemplo de dado estruturado, não módulo exclusivo do produto.

Evidência mínima: Gabaritos de datas, testes de borda e fonte de cada evento.

T07 — Esquecimento operacional

Preparar: Um dado foi confirmado, citado em conversa, resumido e usado numa tarefa.

Executar: Excluir dentro do escopo escolhido e perguntar novamente, também após restart e restauração autorizada.

Resultado exigido: O dado deixa de ser recuperado por índice, histórico ou derivação ativa. Backups/restores respeitam tombstones ou avisam limites antes de reintroduzir conteúdo. Cópias fora do controle são explicitamente diferenciadas.

Evidência mínima: Inspeção de todas as entradas de contexto e linhagem; nenhuma promessa de apagamento físico universal.

T08 — Sensibilidade de ponta a ponta

Preparar: Sentinelas sintéticas SENSITIVE em memória, documento, mensagem e resultado de ferramenta; sem consentimento de nuvem.

Executar: Pedir resumo, resposta e tarefa; depois autorizar apenas uma finalidade e revogar.

Resultado exigido: Antes do consentimento, nenhum transporte/embedding externo recebe sentinela. Depois, somente dados necessários seguem ao destino autorizado. Revogação barra a próxima emissão; guardar memória não autoriza divulgação.

Evidência mínima: Captura de requests em servidor controlado, decisão do egress broker e testes após resumo/restart.

T09 — Mensagem atual nunca descartada

Preparar: Histórico suficiente para exceder o orçamento de contexto; antiga resposta do modelo contém afirmação não comprovada.

Executar: Acrescentar pergunta e correção curtas ao final.

Resultado exigido: Mensagem vigente e restrições ativas entram em toda inferência ou a chamada é bloqueada com motivo. A resposta antiga não é promovida a fato verificado. Fonte resumida continua recuperável.

Evidência mínima: IDs incluídos/excluídos, razões e asserções sobre o payload real.

T10 — Duas perguntas, duas tarefas, assunto novo

Preparar: Duas tarefas independentes aguardam respostas diferentes.

Executar: Responder fora de ordem, perguntar status, iniciar outro assunto e tentar responder questão já resolvida.

Resultado exigido: Cada resposta atinge somente a pergunta correta; status/assunto novo não retoma uma tarefa com conteúdo inadequado. A interface permite escolher a pergunta alvo; a IA só infere quando inequívoco.

Evidência mínima: Testes do contrato reply_to e jornada visual.

T11 — Armazenar, complementar e analisar são ações distintas

Preparar: Um arquivo válido e uma tarefa já aberta.

Executar: Enviar primeiro para guardar; depois como complemento da tarefa; por fim solicitar análise nova.

Resultado exigido: Três intenções produzem vínculos corretos: armazenamento sem tarefa, atualização da existente e nova delegação. A presença de anexo não decide a intenção nem autoriza compartilhamento.

Evidência mínima: Banco, request envelope e contagem de tarefas nos três percursos.

T12 — Documentos reais por formato

Preparar: Mesmo conteúdo sintético em TXT, PDF textual, PDF digitalizado, DOCX, XLSX, PPTX e imagem; incluir tabelas e arquivo corrompido.

Executar: Importar e extrair usando os caminhos reais que o agente utiliza.

Resultado exigido: Valores e cláusulas coincidem com o gabarito; cada trecho tem página/célula/seção. Binário não vira texto UTF-8 artificial. Formato apenas armazenável é rotulado; falha de extração não resulta em análise fabricada.

Evidência mínima: Extratores reais isolados, gabaritos e amostra visual; visão/OCR autorizado quando necessário.

T13 — Informação decisiva no final

Preparar: Documento com cláusulas após 20 mil e 200 mil caracteres e uma tabela final relevante.

Executar: Solicitar comparação integral ou pergunta focada na cláusula final.

Resultado exigido: Leitura paginada alcança a informação. Cobertura fica registrada, fonte continua acessível e nenhuma serialização é cortada no meio. Se a cobertura for insuficiente, isso impede conclusão integral.

Evidência mínima: Ranges lidos, quantidade total, has_more e verificador de cobertura.

T14 — Dez anexos e uploads concorrentes

Preparar: Dez arquivos, mistura de válidos/inválidos, transporte lento e duas conexões.

Executar: Arrastar todos, duplicar chunks, cancelar um upload, interromper antes de finalizar e reiniciar.

Resultado exigido: Cada arquivo tem resultado explícito; nenhum some por estar ocupado. Um upload não duplica bytes, quota e TTL valem entre conexões e a finalização tem recibo recuperável.

Evidência mínima: Hash por arquivo, metadados de upload e limpeza de staging.

T15 — Reenvio imutável e resultado incerto

Preparar: Pedido com texto, anexos e pergunta/tarefa alvo.

Executar: Perder a resposta após gravação, alterar o rascunho na UI e tentar reenviar; repetir na importação.

Resultado exigido: Mesmo ID mantém payload original ou recebe conflito; nova intenção recebe novo ID. Recibo recupera artifact_id criado, sem duplicar importação nem anexar documentos errados.

Evidência mínima: Fault injection em commit/resposta e hash do envelope completo.

T16 — Queda em cada fronteira do processamento

Preparar: Requisição com chave idempotente, intenção faturável e tarefa consequente.

Executar: Injetar crash após RECEIVED, após inferência, antes/depois de criar tarefa e antes de enviar resposta.

Resultado exigido: Reenvio recupera resultado ou continua o processamento incompleto com reconciliação. Não fica duplicate vazio permanente; não repete gasto/efeito incerto sem regra explícita.

Evidência mínima: Matriz de pontos de falha, receipts e uma história de processamento por ID.

T17 — Publicação atômica da tarefa

Preparar: Duas conexões e barreiras que atrasam vínculos de anexos/critério.

Executar: Tentar adquirir a tarefa entre as etapas de preparação.

Resultado exigido: Worker não vê tarefa executável incompleta. A transação final publica objetivo integral, versões, anexos, restrições e critérios juntos; rollback não deixa órfãos executáveis.

Evidência mínima: Teste concorrente nas classes reais e invariantes de banco.

T18 — Pergunta e entrega não desaparecem

Preparar: Tarefa prestes a aguardar usuário ou concluir.

Executar: Falhar na notificação e reiniciar depois do commit de estado.

Resultado exigido: Outbox mantém mensagem pendente e a entrega é reconciliada. Pergunta/resultado aparece uma vez na conversa por chave idempotente; efeito externo não é desfeito só porque a notificação falhou.

Evidência mínima: Estado+outbox na mesma transação, recibos e visualização.

T19 — Correção entre decisão e despacho

Preparar: Worker com decisão antiga pronta; ação ainda não despachada.

Executar: Enviar correção material por outra conexão e liberar a execução suspensa pelo teste.

Resultado exigido: Proposta antiga é recusada pela revisão de instrução/epoch. Próxima decisão usa a correção na mesma tarefa. Ações anteriores confirmadas ficam no histórico e não são repetidas.

Evidência mínima: Barreira determinística, ledger e contexto do próximo passo.

T20 — Parada sob carga pelo controle real

Preparar: Conversa em envio, provedor lento, upload e reconexão simultâneos.

Executar: Acionar o mesmo botão/atalho de parada que o usuário terá.

Resultado exigido: Comando não depende de isSending nem da fila lenta. Nenhum novo despacho após o limite local de projeto, medido; ações já aceitas são descritas como ocorridas/incertas. Parada não gera aprovação.

Evidência mínima: Amostra de latência com hardware, relógios monotônicos e log de despacho; zero bypass.

T21 — Lease longo, worker morto e recuperação

Preparar: Operação dura mais que TTL; em outra execução o worker cai enquanto o processo principal permanece.

Executar: Renovar lease, provocar perda, receber resposta tardia e executar watchdog.

Resultado exigido: Heartbeat independe da inferência. Trabalhador obsoleto não despacha; tarefa não fica RUNNING abandonada e saúde acusa o problema. Agendador não fica paralisado atrás de tarefa longa.

Evidência mínima: Relógio injetável e processos reais; conciliação antes de READY.

T22 — Respostas malformadas e falhas internas

Preparar: Provedor controlado devolve lista, null, campos ausentes, tipos errados, JSON incompleto e esquema extra.

Executar: Executar conversa e tarefa por todos esses casos, inclusive streaming interrompido.

Resultado exigido: Schema completo é validado antes do acesso. Erros são normalizados e limitados; worker não morre silenciosamente; saúde e tarefa refletem falha recuperável. Não aprovar ação com saída inválida.

Evidência mínima: Fuzz/contrato/integração e ausência de efeitos externos.

T23 — Verificação rejeita entrega falsa e aceita português

Preparar: Relatório íntegro sobre assunto errado, valores errados, referência inexistente, cláusula omitida e texto legítimo com a palavra todo.

Executar: Submeter cada resultado ao mesmo verificador usado antes de COMPLETED.

Resultado exigido: Falhas impedem conclusão ou viram entrega parcial identificada. Evidência específica por critério; hash sozinho não basta. Português normal passa e placeholders contextuais verdadeiros falham.

Evidência mínima: Testes negativos e gabaritos de cálculos/fontes/cobertura.

T24 — Orçamento reduzido durante o trabalho

Preparar: Tarefa de vários passos com orçamento inicialmente alto.

Executar: Reduzir teto e revogar consentimento entre dois despachos; executar chat/voz/teste em paralelo.

Resultado exigido: Próxima reserva respeita revisão atual, não snapshot antigo. Consumo já ocorrido e reservas em voo permanecem auditáveis. Não há nova chamada incompatível após a alteração ser efetivada.

Evidência mínima: Concorrência real no Core, extrato de reservas e revisão de política.

T25 — Cobrança e efeito desconhecidos

Preparar: Provedor/serviço controlado processa e perde a resposta, ou falha antes de receber.

Executar: Simular timeout, crash e retorno sem usage; depois reconciliar com evidência.

Resultado exigido: Não enviado comprovado libera reserva; incerto permanece UNKNOWN; custo confirmado liquida. Retry não duplica compra/envio e estimativa não aparece como valor faturado.

Evidência mínima: Ledger antes/depois, recibos controlados e aprovações reservadas.

T26 — Credenciais não seguem redirecionamento indevido

Preparar: Servidores de teste em origens distintas e segredo exclusivamente sintético.

Executar: Exercitar 301/302/303/307/308, downgrade, userinfo, portas e host com prefixo parecido com loopback.

Resultado exigido: Destino não autorizado nunca recebe Authorization, cookies ou token. HTTP só em loopback exato habilitado para teste. Repetir com o Python efetivamente empacotado.

Evidência mínima: Captura no destino de teste, análise de URL e decisão de bloqueio.

T27 — Isolamento de objetos e sessões

Preparar: Duas identidades sintéticas com memórias, mensagens, fontes e artefatos próprios.

Executar: Tentar delegar/corrigir/ler um objeto usando IDs do outro escopo.

Resultado exigido: Toda operação falha sem revelar conteúdo. Verificações relacionam sessão, proprietário, empregado e objeto; não basta role=owner. Dispositivo remoto revogado não recupera token local.

Evidência mínima: Matriz de autorização positiva e negativa em todos os métodos expostos.

T28 — Duas instâncias não disputam autoridade

Preparar: Mesmo diretório de dados, dois Supervisores ou duas cópias do app iniciados simultaneamente.

Executar: Fazer boot concorrente e depois deixar arquivos de socket órfãos de uma queda.

Resultado exigido: Há um único responsável por tokens, banco de autoridade e recuperação. Segunda janela conecta ou recebe diagnóstico; não apaga socket saudável. Limpeza distingue processo vivo de arquivo órfão.

Evidência mínima: Trava interprocesso, PID/epoch e testes de retomada.

T29 — Troca de chave invalida inteligência

Preparar: Chave sintética A validada para modelo/endpoint; chave B sem acesso.

Executar: Trocar chave, endpoint, modelo e versão de capacidade separadamente.

Resultado exigido: Indicador Pronta é removido quando seu vínculo deixa de valer. Preço, acesso e qualidade continuam estados distintos. Novo teste pago precisa de autorização e entra no ledger global.

Evidência mínima: Validações vinculadas e UI atualizada.

T30 — Controles coerentes com cada estado

Preparar: Tarefas em todos os estados, inclusive BLOCKED por orçamento e UNKNOWN externo.

Executar: Usar pausa, retomada, desbloqueio, rejeição e cancelamento pela UI.

Resultado exigido: Só ações válidas aparecem; orçamento liberado reavalia condição antes de executar. UNKNOWN não vira reenvio cego para fazer o botão funcionar. Estados não elegíveis não ficam sem caminho explicável.

Evidência mínima: Tabela de transições, app real e asserções do Core.

T31 — Encerramento responsivo e disponibilidade verdadeira

Preparar: Serviço de teste ignora encerramento cooperativo; uma operação externa está incerta.

Executar: Fechar janela, pausar, encerrar serviços, sair, suspender e reiniciar o Mac.

Resultado exigido: Cada operação tem significado correto. Shutdown fora da UI tem prazo e diagnóstico; pendências são reconciliadas. Não prometer trabalhar desligado ou antes do desbloqueio necessário.

Evidência mínima: Jornada Mac e processos, logs redigidos e casos de suspensão/login.

T32 — Arquivos grandes e exportação útil

Preparar: Artefatos imutáveis de 1, 10 e 50 MiB em formatos suportados.

Executar: Ler por chunks, adulterar fonte em um teste e salvar sem mudar nome sugerido.

Resultado exigido: Custo de leitura/hash cresce aproximadamente linearmente; adulteração é recusada; nome/extensão correspondem ao conteúdo e o arquivo abre. Não remover integridade para acelerar.

Evidência mínima: Instrumentação de I/O, hashes e abertura com aplicativo compatível.

T33 — Computador próprio realmente isolado

Preparar: VM de trabalho e Execution Box com código de teste tentando host, rede local, metadados e cofre.

Executar: Executar tentativas por arquivos, DNS, IPv4/IPv6, UDP e acesso direto fora do proxy.

Resultado exigido: Sentinelas do host são inacessíveis; saída proibida falha inclusive fora do browser. Um perfil de Chrome e um processo Python separado não contam como a VM. Sem fallback para desktop pessoal.

Evidência mínima: Mac com virtualização real, threat model e registros do mediador externo.

T34 — Browser, obstáculo e takeover

Preparar: Navegador no guest; site de teste com login, CAPTCHA simulado e formulário material.

Executar: Pesquisar, encontrar bloqueio, pedir intervenção, usuário assumir e devolver.

Resultado exigido: Só um controlador opera; cliques antigos são descartados; segredo digitado não vira log/screenshot persistente. IA tenta alternativa autorizada, não promete acesso universal nem contorna controle.

Evidência mínima: Tela real, evento de takeover, nova observação e ausência de ações atrasadas.

T35 — Pesquisa pública com fonte e data

Preparar: Objetivo geral exige informações atuais de mais de uma fonte.

Executar: Pesquisar no guest/adaptador real, comparar e gerar entrega com referências.

Resultado exigido: Fontes foram de fato recuperadas, datas e trechos sustentam afirmações; divergências são expostas. Não usar preço em cache como cotação ao vivo nem declarar consulta realizada se só houve tentativa.

Evidência mínima: URLs, captured_at, evidência mínima, artefato e critérios cumpridos.

T36 — Conta operacional e recurso pago

Preparar: Conta dedicada sob responsabilidade do dono e sandbox de serviço com assinatura/compra.

Executar: Propor recurso, receber aprovação limitada, mudar preço/destinatário e perder resposta após envio.

Resultado exigido: Sem autorização aplicável não há compromisso; mudança material exige nova aprovação; UNKNOWN é conciliado. Cadastro não inventa identidade. Entrega usa conta do Atlas, não e-mail pessoal por padrão.

Evidência mínima: Cartão de aprovação, escopos, recibos e testes sem compras reais não autorizadas.

T37 — Aprender uma skill sem alterar suas próprias regras

Preparar: Tarefa repetida e skill candidata gerada com e sem tentativa de rede/cofre.

Executar: Testar em Execution Box, comparar com baseline, promover sob política e forçar regressão.

Resultado exigido: Procedimento útil pode ser reutilizado com versão/evidência; candidato malicioso é bloqueado; ampliação de acesso exige aprovação. Rollback restaura versão anterior sem apagar auditoria.

Evidência mínima: Manifesto, corpus de teste, provenance, promoção e rollback reais.

T38 — Voz, correções e interrupções

Preparar: Português brasileiro com ruído, nomes próprios, datas e valores semelhantes.

Executar: Falar, ver transcrição, interromper a fala e depois mandar parar a tarefa.

Resultado exigido: Mesmo funcionário e contexto do texto; cancelar TTS não equivale automaticamente a cancelar tarefa. Número/destinatário incerto é confirmado; áudio ambiente não autoriza pagamento. Microfone e custo são visíveis.

Evidência mínima: Teste com áudio consentido/sintético e dispositivos reais; nenhuma gravação contínua escondida.

T39 — Celular, pareamento, offline e revogação

Preparar: Companion real, relay e Mac pareados; chaves de teste próprias.

Executar: Enviar objetivo, ficar offline, reconectar, receber artefato, revogar dispositivo e tentar replay.

Resultado exigido: Relay recebe ciphertext sob protocolo validado; não afirma trabalho começou offline. Uma tarefa/entrega por ID; revogado não lê nem comanda; aprovação sensível exige interface autenticada.

Evidência mínima: Dois dispositivos, rede adversa, filas duráveis, teste criptográfico do protocolo adotado e captura da UI.

T40 — Aceite completo, backup e atualização

Preparar: Build candidato, dados sintéticos, Mac limpo compatível e usuário sem programação.

Executar: Instalar, criar identidade, conversar, lembrar, pesquisar, executar no guest, aprovar recurso, falar, receber pelo celular; atualizar, restaurar e excluir.

Resultado exigido: Jornada termina sem terminal, com evidência e capacidades verdadeiras. Migração preserva conteúdo; restore não repete efeitos nem restaura aprovações perigosas. Nenhuma lacuna essencial é escondida pela etiqueta V1.

Evidência mínima: Pacote assinado, versão/hash, gravação real, matriz T01-T40, custos e aceite humano informado.

ANEXO C — 24 pacotes de implementação até a V1

Estes pacotes completam a visão, além de reparar os 32 achados. Cada pacote deve ser fatiado em mudanças revisáveis. Seus caminhos são alvos de integração propostos: reaproveitar equivalentes existentes e não criar dois serviços para a mesma responsabilidade. Dependências listadas são mínimas; o gate também exige as correções correspondentes do Anexo A. Nada nesta lista foi implementado ou testado por esta entrega documental.

Não tratar N24 como um novo planejamento: é a execução dos testes, a revisão e a entrega do que já foi construído. Recursos humanos/credenciais ausentes devem bloquear somente sua integração/teste dependentes; implementar componentes não bloqueados em paralelo, sem falsificar aprovação do gate.

N01 — Adotar contrato, baseline e rastreabilidade

Gate: G0. **Capítulos normativos:** 0, 26–28. **Dependências mínimas:** baseline existente.

Pontos de implementação: docs/spec/; docs/PROGRESS.md; docs/REQUIREMENTS_TRACEABILITY.md.

Execução exigida: Registrar HEAD/estado local, ler contrato e auditoria, executar o que o ambiente permite e decompor os itens sem refazer o projeto. Criar evidência por achado e manter status real; obter primeiro teste de regressão nas classes existentes.

Aceite de G0: baseline registrada, documentos lidos, matriz criada e primeira regressão pertinente executada. A instalação completa de T01 pertence a N23/N24, não é pré-requisito deste pacote inicial.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N02 — Authority Core e transporte de controle

Gate: G1. **Capítulos normativos:** 3–6, 9, 12. **Dependências mínimas:** N01.

Pontos de implementação: core/; security/broker/; security/policy/; shared/schemas/.

Execução exigida: Separar interface, propostas e autoridade; introduzir versões/epochs e lane de controle independente. Todas as emissões e operações de efeito validam a política vigente no broker; runtime e guest não escrevem permissões.

Aceite vinculado: T08, T19, T20, T27. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N03 — Inbox, transações e outbox recuperáveis

Gate: G1. **Capítulos normativos:** 4–6. **Dependências mínimas:** N01.

Pontos de implementação: storage/migrations/; storage/repositories/; core/service.py; core/conversation.py.

Execução exigida: Persistir envelope canônico e hash, estados de processamento e recibo. Publicar tarefa/anexos/critério atômicamente. Outbox no commit de estado; deduplicar localmente e reconciliar efeitos externos sem prometer exactly-once universal.

Aceite vinculado: T15, T16, T17, T18. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N04 — Supervisão, leases e agenda independente

Gate: G1. **Capítulos normativos:** 6, 22. **Dependências mínimas:** N02, N03.

Pontos de implementação: core/daemon.py; runtime/tasks/; platform/macros/AtlasKit/.

Execução exigida: Executar heartbeat/watchdog/scheduler fora do bloqueio do worker, recuperar leases abandonados e reportar saúde real. Trava de instância antes de tokens/sockets; shutdown/restart limitados e controle claro do ciclo de vida.

Aceite vinculado: T21, T22, T28, T31. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N05 — Cofre, egress e orçamento dinâmico

Gate: G1. **Capítulos normativos:** 9, 11–12. **Dependências mínimas:** N02, N03.

Pontos de implementação: security/vault/; security/budget/; security/network/; core/intelligence.py.

Execução exigida: Preservar classificação e fonte até o payload final. Conferir consentimento/destino e orçamento dentro de cada emissão. Bind da validação à chave/modelo/endpoint; ledger distingue reservado, real, estimado e desconhecido.

Aceite vinculado: T08, T24, T25, T26, T29. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N06 — Contexto comum e conhecimento duradouro

Gate: G2. **Capítulos normativos:** 7–9. **Dependências mínimas:** N02, N05.

Pontos de implementação: runtime/memory/; runtime/models/context.py; core/conversation.py.

Execução exigida: Criar recuperação híbrida com proveniência e vigência compartilhada por chat e tarefas. Reservar contexto obrigatório; sumarização conserva origem e sensibilidade. Implementar linhagem/tombstones e tela de gestão para correção/esquecimento.

Aceite vinculado: T04, T05, T07, T09. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N07 — Intenções, perguntas e continuidade

Gate: G2. **Capítulos normativos:** 5, 7, 21. **Dependências mínimas:** N03, N06.

Pontos de implementação: core/conversation.py; core/service.py; platform/macros/AtlasKit/.

Execução exigida: Usar contratos de intenção e alvo explícito. Comandos de autoridade são determinísticos; assunto novo não responde uma pergunta por acidente. Anexo pode ser armazenado, vinculado ou analisado; correção afeta tarefa vigente sem nova delegação.

Aceite vinculado: T03, T10, T11, T19. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N08 — Documentos e extração por formato

Gate: G2. **Capítulos normativos:** 10, 14. **Dependências mínimas:** N03, N05.

Pontos de implementação: runtime/artifacts/; runtime/tools/; platform/execution/.

Execução exigida: Implementar extratores reais e isolados, incluindo texto, tabelas, páginas/células e fonte visual quando exigida. Registrar cobertura e segmentos completos com paginação. Formato não suportado fica armazenável, não analisado artificialmente.

Aceite vinculado: T12, T13. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N09 — Datas, unidades e cálculos verificáveis

Gate: G2. **Capítulos normativos:** 8, 10, 13. **Dependências mínimas:** N06, N08.

Pontos de implementação: runtime/tools/; shared/; tests/evals/.

Execução exigida: Usar código para datas, intervalos, recorrências, unidades, moedas e números. Derivar valores das fontes e guardar rastreabilidade. Não criar calendário de filhos como produto à parte; oferecer operações estruturadas generalistas.

Aceite vinculado: T06, T23. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N10 — Planner, contratos de ferramenta e critérios

Gate: G2. **Capítulos normativos:** 6, 11, 13. **Dependências mínimas:** N02, N06, N08.

Pontos de implementação: runtime/agent/; runtime/tools/registry.py; runtime/verification/.

Execução exigida: Preservar pedido completo e gerar plano com critérios rastreáveis. Validar schema completo antes de uso, catálogos habilitados apenas e evidência por critério. Corrigir falso positivo de todo. Não confundir sucesso estrutural com atendimento factual.

Aceite vinculado: T19, T22, T23. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N11 — Artefatos, uploads e exportação

Gate: G3. **Capítulos normativos:** 10, 14, 21. **Dependências mínimas:** N03, N08.

Pontos de implementação: runtime/artifacts/; core/service.py; platform/macros/AtlasKit/.

Execução exigida: Implementar filas por arquivo, recibo da importação, quota/TTL globais e leitura linear de objeto imutável. Exportar formatos reais com nome/extensão e render/abertura, preservando original e impedindo path escape.

Aceite vinculado: T14, T15, T32. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N12 — Interface local completa e reconciliação

Gate: G3. **Capítulos normativos:** 7, 21–22. **Dependências mínimas:** N04, N07, N11.

Pontos de implementação: platform/macros/AtlasKit/Sources/AtlasApp/;
platform/macros/AtlasKit/Sources/AtlasKit/.

Execução exigida: Conectar as telas aos estados reais, fazer upsert/cursor de histórico, anexos/respostas vinculados, botões derivados do estado e retomada de bloqueios. Manter UI responsiva e parada fora do fluxo de envio. Testar app, não só ViewModel.

Aceite vinculado: T03, T10, T30, T31. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N13 — Homologar inteligência e modos

Gate: G4. **Capítulos normativos:** 11–12, 25. **Dependências mínimas:** N05, N10, N12.

Pontos de implementação: runtime/models/; core/intelligence.py; tests/evals/.

Execução exigida: Validar IDs/capacidades atuais e acesso da conta com autorização. Executar tarefa geral real no bundle, medir qualidade/custo e provar que modos mudam seleção. Defaults de modelo são candidatos, não acesso ou preço garantidos.

Aceite vinculado: T04, T19, T20, T23, T24, T29. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N14 — Workspace Linux ARM64 e mediação

Gate: G5. **Capítulos normativos:** 3, 15, 22. **Dependências mínimas:** N02, N04, N05.

Pontos de implementação: platform/workspace/; platform/macros/; security/network/.

Execução exigida: Provar VM por Virtualization.framework, guest agent, transporte e egress externo. Não montar home nem liberar NIC irrestrita. Arquitetura virtio socket/proxy ou alternativa justificada por ADR precisa funcionar e bloquear sentinelas no Mac real.

Aceite vinculado: T02, T33. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N15 — Execution Box e código gerado

Gate: G5. **Capítulos normativos:** 15, 17. **Dependências mínimas:** N14.

Pontos de implementação: platform/execution/; security/policy/; runtime/tools/.

Execução exigida: Executar código novo em ambiente descartável sem tokens/cofre/dados de produção e sem rede padrão. Impor tempo/CPU/memória/disco. Exportar somente artefatos validados. Processo filho sozinho não equivale a sandbox.

Aceite vinculado: T33, T37. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N16 — Browser, pesquisa e tela de trabalho

Gate: G5. **Capítulos normativos:** 13, 15–16. **Dependências mínimas:** N13, N14, N15.

Pontos de implementação: platform/workspace/; runtime/tools/; platform/macros/AtlasKit/.

Execução exigida: Empacotar Playwright/browser compatíveis, APIs antes de cliques livres, perfis separados e ação sensível por driver homologado. Mostrar tela real/takeover com exclusividade e invalidar cliques antigos. Pesquisa entrega fontes recuperadas e atualizadas.

Aceite vinculado: T34, T35. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N17 — Identidade operacional, contas e e-mail

Gate: G6. **Capítulos normativos:** 12, 18. **Dependências mínimas:** N05, N16.

Pontos de implementação: runtime/connectors/; security/vault/; core/service.py.

Execução exigida: Criar percurso de conta dedicada administrada pelo dono; OAuth/recuperação/termos visíveis. Entrega ao canal verificado sob mandato. Remetente de e-mail não é autenticador de comando. Implementar recibos e reconciliação.

Aceite vinculado: T36. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N18 — Solicitações de recursos e ações materiais

Gate: G6. **Capítulos normativos:** 12, 16, 18. **Dependências mínimas:** N17.

Pontos de implementação: security/approvals/; runtime/connectors/; platform/macros/AtlasKit/.

Execução exigida: Proposta de serviço inclui necessidade, preço consultado, renovação, dados e alternativas. Executar compromisso somente com mandato/aprovação aplicáveis. Sandbox de pagamento primeiro; cartão limitado não elimina autorização nem reconciliação.

Aceite vinculado: T25, T36. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N19 — Aprendizado e ciclo de skills

Gate: G6. **Capítulos normativos:** 8, 17. **Dependências mínimas:** N10, N15.

Pontos de implementação: runtime/skills/; tests/evals/; storage/migrations/.

Execução exigida: Persistir lições resumidas ligadas a evidência; criar manifestos e testes de skill, quarentena/promoção/revogação/rollback. Autopromoção só no mesmo conjunto de direitos e sob política prévia. Nenhuma skill altera Core ou segurança.

Aceite vinculado: T37. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N20 — Conversa por voz

Gate: G7. **Capítulos normativos:** 11-12, 20. **Dependências mínimas:** N07, N13.

Pontos de implementação: runtime/voice/; platform/macos/AtlasKit/.

Execução exigida: Integrar transcrição/síntese/streaming ao mesmo contexto e Task Engine; microfone explícito, retenção mínima, barge-in e confirmações de valores. Escolher modelo de voz documentado e testar pt-BR; cobrar no mesmo ledger.

Aceite vinculado: T38. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N21 — Companion remoto e canal seguro

Gate: G7. **Capítulos normativos:** 5, 9, 19. **Dependências mínimas:** N03, N05, N12.

Pontos de implementação: apps/companion-web/; gateway/; shared/schemas/.

Execução exigida: Pairing com chave por dispositivo, relay durável de ciphertext via protocolo estabelecido, conexão de saída do Mac, replay/revogação/ACKs e fila offline. Não criar outro núcleo ou expor portas administrativas. E-mail adicional e WhatsApp oficial condicional.

Aceite vinculado: T18, T39. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N22 — Onboarding leigo, privacidade e recuperação

Gate: G8. **Capítulos normativos:** 9, 21-24. **Dependências mínimas:** N12, N14, N17, N20, N21.

Pontos de implementação: platform/macos/AtlasKit/; storage/backup/; security/; docs/.

Execução exigida: Completar identidade, modos, orçamento, contas e disponibilidade em português claro. Cifrar banco/artefatos/índices/staging com bibliotecas auditadas e chaves recuperáveis autorizadas. Item de login, acessibilidade, exportação/exclusão e restore com revogações.

Aceite vinculado: T01, T02, T07, T31, T40. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N23 — Build, atualização e distribuição

Gate: G9. **Capítulos normativos:** 22-24, 28. **Dependências mínimas:** N22.

Pontos de implementação: packaging/; .github/workflows/; scripts/; docs/.

Execução exigida: Empacotar dependências/VM verificadas, SBOM/licenças, certificados fora do app/agente, assinatura/notarização e instalador. Update compatível com DB/workspace/skills e rollback controlado. Gates críticos sem continue-on-error.

Aceite vinculado: T01, T31, T40. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

N24 — Aceite integrado e entrega V1

Gate: G10. **Capítulos normativos:** 24–28. **Dependências mínimas:** N18, N19, N23.

Pontos de implementação: tests/; docs/evidence/; docs/ACCEPTANCE_REPORT.md.

Execução exigida: Revisar controles independentemente, executar cenários no bundle com serviços/modelos reais autorizados e usuário comum. Entregar matriz rastreável, custos, riscos e manual. Publicação final só com aceite humano; pendência essencial impede chamar V1 completa.

Aceite vinculado: T01, T02, T03, T04, T05, T06, T07, T08, T09, T10, T11, T12, T13, T14, T15, T16, T17, T18, T19, T20, T21, T22, T23, T24, T25, T26, T27, T28, T29, T30, T31, T32, T33, T34, T35, T36, T37, T38, T39, T40. Demonstrar o percurso integrado correspondente e os controles negativos; teste sem acesso ao recurso real não homologa esse recurso.

Entrega do pacote: código e migrações pertinentes, schemas/documentação sincronizados, regressões, comandos/resultados, versões e evidências. Atualizar requisito → implementação → teste → evidência. Declarar o que ainda depende de Mac, provedor, conta, assinatura ou aceite humano.

Encerramento do contrato

O próximo passo do agente programador é trabalhar no repositório: revalidar a baseline, registrar a matriz, iniciar a primeira correção de controle com seu teste e continuar pelas dependências. O produto final continua sendo o funcionário digital completo descrito neste contrato; nenhum marco intermediário o substitui.